ntic master

UNIVERSIDAD COMPLUTENSE MADRID

MÓDULO

Bases de Datos SQL

PROFESOR:

Juan Fernando Sánchez Martínez

ntic master

UNIVERSIDAD COMPLUTENSE MADRID

Índice de Contenidos

1. Unit 1. Introducción a las Bases de Datos
2. Unit 2. Modelado Bases de Datos
3. Unit 3. Gestión de Bases de Datos
4. Unit 4. Lenguaje SQL
    a. DDL (Data Definition Language)
    b. DML (Data Manipulation Language)



Bases de datos relacionales

Este capítulo cubre los conceptos básicos de las bases de datos relacionales, explicando cómo organizan la información en tablas y las ventajas clave de este enfoque. También introduce el lenguaje SQL y los componentes esenciales de una base de datos relacional, como las tablas, claves primarias, foráneas e índices.

ntic master

UNIVERSIDAD COMPLUTENSE MADRID

1. Unit 1. Introducción a las Bases de Datos

1.1 Bases de Datos Relacionales

1.2 Introducción a SQL

1.3 Estructura de Tablas

1 Bases de datos relacionales: Introducción

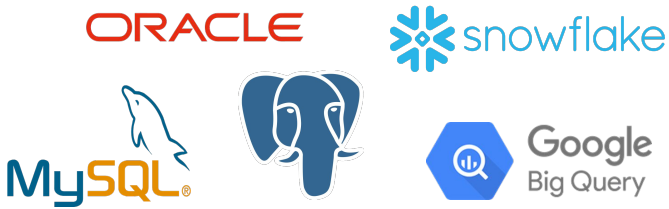
Bases de datos relacionales (SQL)

Definición (Oracle):

- Colección organizada de información estructurada.
- Controlada por un **Sistema de Gestión de Bases de Datos (SGBD)**.
- SGBD + datos + aplicaciones asociadas = **Sistema de Base de Datos**.

Modelo Relacional:

- Los datos se organizan en **filas y columnas** dentro de tablas.
- Facilita el procesamiento y consulta eficiente de datos.



1 Bases de datos relacionales: Introducción

Definición:

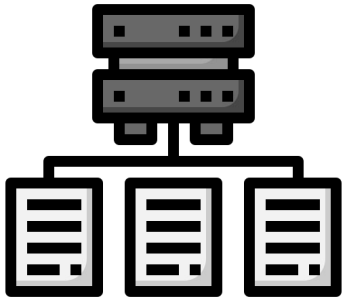
- Colección organizada de información estructurada.
- Controlada por un **SGBD** (Sistema de Gestión de Bases de Datos).
- **SGBD + datos + aplicaciones** = **Sistema de Base de Datos**.

Acceso a los Datos:

- Usamos el **lenguaje SQL** para consultar y combinar datos almacenados en diferentes tablas.

Ventajas frente a archivos planos:

- Gestión centralizada mediante **SGBD**.
- **Capas de abstracción** de datos.
- **Independencia física** y **lógica** de los datos.
- Acceso **simultáneo** por múltiples usuarios.
- Mayor **seguridad** y **gobernabilidad**.



1 Bases de datos: Lenguaje SQL

Definición:

- SQL significa **Structured Query Language**.
- Es el lenguaje más utilizado para **crear, consultar y actualizar** bases de datos relacionales.

Historia y Estándares:

- Desarrollado en la década de 1970 basado en el **modelo relacional de datos**.
- Originalmente llamado **SEQUEL** (Structured English Query Language), abreviado luego a **SQL**.
- Estándar internacional desde 1986, cuando fue adoptado por **ANSI (American National Standards Institute)**.
- Versiones clave del estándar ANSI:
  - **SQL-86**: Primera versión del estándar.
  - **SQL-89**: Revisión menor que corrige errores.
  - **SQL-92**: Estándar ampliado con nuevas funciones (subconsultas, operadores como `JOIN`).
  - **SQL:1999**: Introducción de **triggers**, **procedimientos almacenados**, y soporte para **objetos**.
  - **SQL:2003**: Añade soporte para **XML** y tipos de datos autodefinidos.
  - **SQL:2008**: Mejora la gestión de **datos escalares** y columnas calculadas.
  - **SQL:2011**: Introduce mejoras en **manejo de datos temporales**.
  - **SQL:2016**: Añade soporte para **JSON**.
  - **SQL:2019**: Mejora el soporte para **big data** y el uso de **polimorfismo de tablas**.

1 Bases de datos: Lenguaje SQL

Categorías de Instrucciones SQL

SQL se divide en difernrtes categorías principales, dependiendo de la función que realizan. En este módulo estudiamos 2:

a) Lenguaje de Definición de Datos (DDL)

- Usado para **crear, modificar o eliminar** objetos en la base de datos (tablas, esquemas, vistas, procedimientos).
- Palabras clave:
  - **CREATE**: Crea un objeto (ej. `CREATE TABLE`).
  - **ALTER**: Modifica un objeto existente (ej. `ALTER TABLE`).
  - **DROP**: Elimina un objeto (ej. `DROP TABLE`).

b) Lenguaje de Manipulación de Datos (DML)

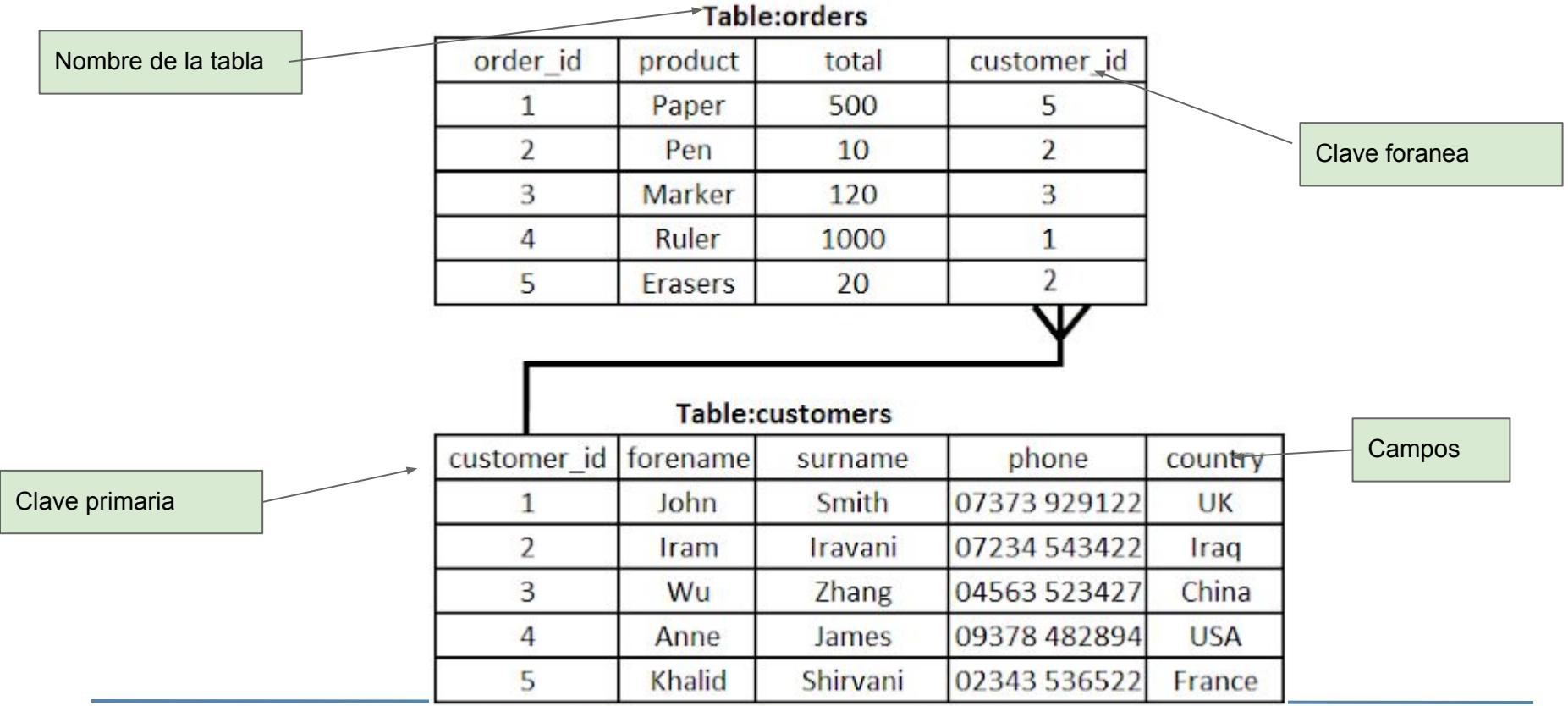
- Usado para **manipular los datos** dentro de la base de datos.
- Palabras clave:
  - **SELECT**: Recupera datos de las tablas.
  - **INSERT**: Añade nuevos datos.
  - **UPDATE**: Modifica datos existentes.
  - **DELETE**: Borra datos.

1 Bases de datos: Tablas

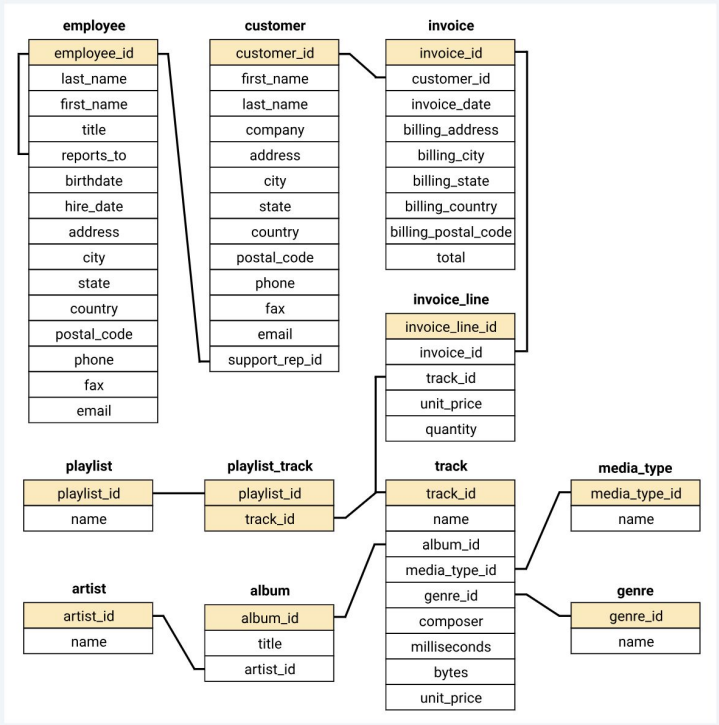
Conceptos Clave al Definir Tablas en SQL

- **Nombres de Tablas**:
  - Deben estar en **minúsculas** y no incluir espacios.
  - Usar **guiones bajos** (_) en lugar de espacios (ej. `clientes_activos`).
  - El nombre de la tabla debe referirse a un **grupo colectivo** (ej. `clientes`).
- **Nombres de Campos**:
  - Deben estar en **minúsculas** y no contener espacios.
  - Deben ser **singulares** ya que describen un dato de un solo registro (ej. `nombre`, `apellido`).
  - Dos campos en la misma tabla **no pueden tener el mismo nombre**.
- **Diferenciación entre Tabla y Campos**:
  - El nombre de un campo **nunca debe coincidir** con el de la tabla para evitar confusión al referirse a cada uno (ej. evitar `cliente.cliente_id`).
- **Clave Primaria (Primary Key)**:
  - Es un valor **único** que identifica de forma exclusiva un registro en la tabla.
  - Ejemplo: en la tabla `clientes`, el campo `id` será la clave primaria que identifica cada cliente.
- **Clave Foránea (Foreign Key)**:
  - Es un campo que **hace referencia** a la **clave primaria** de otra tabla.
  - Las claves foráneas son las preferidas para **relacionar** tablas a través de combinaciones en consultas SQL.

1 Bases de datos: Tablas



1 Bases de datos: Tablas





Modelado Bases de Datos

En este capítulo se aborda el proceso de diseño de bases de datos relacionales, empezando por el diseño conceptual (modelo entidad-relación), pasando por el diseño lógico (creación de tablas y relaciones) y finalizando con el diseño físico (implementación en un sistema de gestión de bases de datos).

ntic master

UNIVERSIDAD COMPLUTENSE MADRID

2. Unit 2. Modelado Bases de Datos

2.1. Fundamentos del Modelado

2.2. Análisis de Requisitos

2.3. Modelo Conceptual

2.4. Modelo Lógico

2.5. Modelo Físico

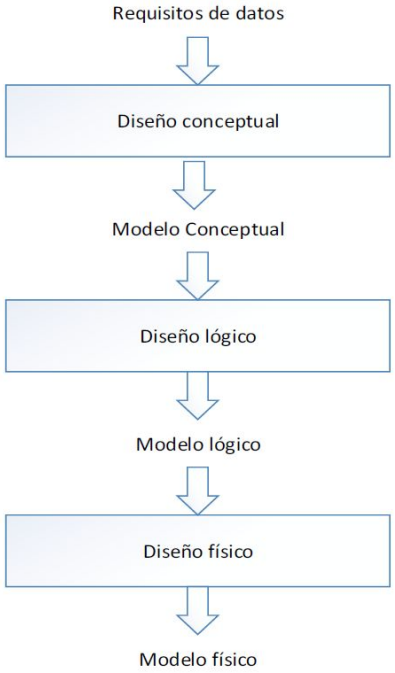
2 Modelado de Bases de Datos: Fundamentos

Creación de Bases de Datos en Aplicaciones Empresariales

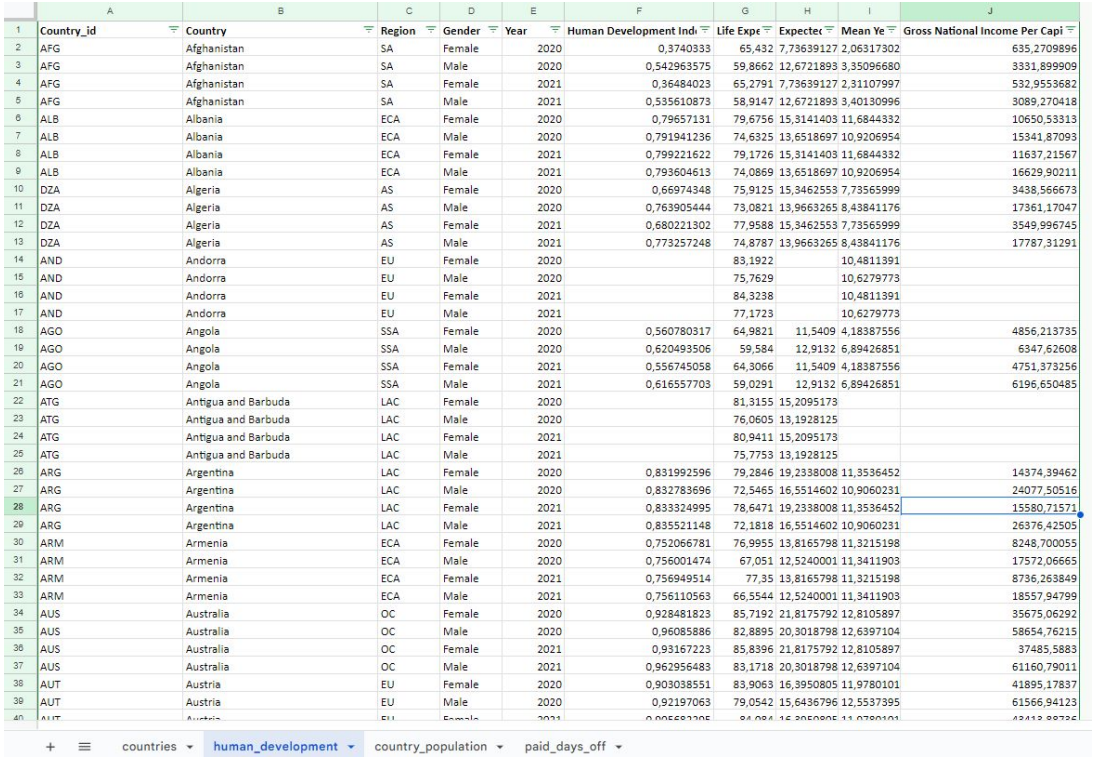
- **Fundamental para aplicaciones empresariales**:
  - Las bases de datos almacenan información crítica para el funcionamiento de aplicaciones.
  - Utilizadas mediante un **SGBD** disponible en el mercado.

- **Diseño de bases de datos**:
  - Realizado por **diseñadores de bases de datos** profesionales

- **Metodología recomendada**:
  - El diseño sigue una serie de **pasos estructurados** que garantizan eficiencia y organización.



2 Modelado de Bases de Datos: Fundamentos



	A	B	C	D	E	F	G	H	I	J
1	Country_id	Country	Region	Gender	Year	Human Development Ind	Life Expe	Expectec	Mean Ye	Gross National Income Per Capi
2	AFG	Afghanistan	SA	Female	2020	0,3740333	65,432	7,73639127	2,06317302	635,2709896
3	AFG	Afghanistan	SA	Male	2020	0,542963575	59,8662	12,6721893	3,35096680	3331,899909
4	AFG	Afghanistan	SA	Female	2021	0,36484023	65,2791	7,73639127	2,31107997	532,9553682
5	AFG	Afghanistan	SA	Male	2021	0,535610873	58,9147	12,6721893	3,40130996	3089,270418
6	ALB	Albania	ECA	Female	2020	0,79657131	79,6756	15,3141403	11,6844332	10650,53313
7	ALB	Albania	ECA	Male	2020	0,791941236	74,6325	13,6518697	10,9206954	15341,87093
8	ALB	Albania	ECA	Female	2021	0,799221622	79,1726	15,3141403	11,6844332	11637,21567
9	ALB	Albania	ECA	Male	2021	0,793604613	74,0869	13,6518697	10,9206954	16629,90211
10	DZA	Algeria	AS	Female	2020	0,66974348	75,9125	15,3462553	7,73565999	3438,566673
11	DZA	Algeria	AS	Male	2020	0,763905444	73,0821	13,9663265	8,43841176	17361,17047
12	DZA	Algeria	AS	Female	2021	0,680221302	77,9588	15,3462553	7,73565999	3549,996745
13	DZA	Algeria	AS	Male	2021	0,773257248	74,8787	13,9663265	8,43841176	17787,31291
14	AND	Andorra	EU	Female	2020		83,1922		10,4811391	
15	AND	Andorra	EU	Male	2020		75,7629		10,6279773	
16	AND	Andorra	EU	Female	2021		84,3238		10,4811391	
17	AND	Andorra	EU	Male	2021		77,1723		10,6279773	
18	AGO	Angola	SSA	Female	2020	0,560780317	64,9821	11,5409	4,18387556	4856,213735
19	AGO	Angola	SSA	Male	2020	0,620493506	59,584	12,9132	6,89426851	6347,62608
20	AGO	Angola	SSA	Female	2021	0,556745058	64,3066	11,5409	4,18387556	4751,373256
21	AGO	Angola	SSA	Male	2021	0,616557703	59,0291	12,9132	6,89426851	6196,650485
22	ATG	Antigua and Barbuda	LAC	Female	2020		81,3155	15,2095173		
23	ATG	Antigua and Barbuda	LAC	Male	2020		76,0605	13,1928125		
24	ATG	Antigua and Barbuda	LAC	Female	2021		80,9411	15,2095173		
25	ATG	Antigua and Barbuda	LAC	Male	2021		75,7753	13,1928125		
26	ARG	Argentina	LAC	Female	2020	0,831992596	79,2846	19,2338008	11,3536452	14374,39462
27	ARG	Argentina	LAC	Male	2020	0,832783696	72,5465	16,5514602	10,9060231	24077,50516
28	ARG	Argentina	LAC	Female	2021	0,833324995	78,6471	19,2338008	11,3536452	15580,71571
29	ARG	Argentina	LAC	Male	2021	0,835521148	72,1818	16,5514602	10,9060231	26376,42505
30	ARM	Armenia	ECA	Female	2020	0,752066781	76,9955	13,8165798	11,3215198	8248,700055
31	ARM	Armenia	ECA	Male	2020	0,756001474	67,051	12,5240001	11,3411903	17572,06665
32	ARM	Armenia	ECA	Female	2021	0,756949514	77,35	13,8165798	11,3215198	8736,263849
33	ARM	Armenia	ECA	Male	2021	0,756110563	66,5544	12,5240001	11,3411903	18557,94799
34	AUS	Australia	OC	Female	2020	0,928481823	85,7192	21,8175792	12,8105897	35675,06292
35	AUS	Australia	OC	Male	2020	0,96085886	82,8895	20,3018798	12,6397104	58654,76215
36	AUS	Australia	OC	Female	2021	0,93167223	85,8396	21,8175792	12,8105897	37485,5883
37	AUS	Australia	OC	Male	2021	0,962956483	83,1718	20,3018798	12,6397104	61160,79011
38	AUT	Austria	EU	Female	2020	0,903038551	83,9063	16,3950805	11,9780101	41895,17837
39	AUT	Austria	EU	Male	2020	0,92197063	79,0542	15,6436796	12,5537395	61566,94123
40	AUT	Austria	EU	Female	2021	0,905682205	84,084	16,3950805	11,9780101	43413,88736

countries | human_development | country_population | paid_days_off

2 Modelado de Bases de Datos: Diseño Conceptual

Creación del Modelo Conceptual

- **Objetivo**: Reflejar los **requisitos de datos**.
  - Incluye: **Entidades**, **Relaciones** y **Restricciones**.
- **Independencia del modelo lógico**:
  - Puede transformarse en cualquier modelo lógico: **Jerárquico**, **En red**, **Relacional** u **Orientado a Objetos**.
  - Independiente del **SGBD** utilizado para la implementación.
- **Ventajas**:
  - Facilita la captura de propiedades de los datos sin preocuparse por el almacenamiento.
  - Focaliza en un buen diseño conceptual.
- **Modelo Entidad-Relación**:
  - El **modelo conceptual** más utilizado para representar los datos.

2 Modelado de Bases de Datos: Diseño Conceptual

Para cada relación podemos definir sus **propiedades**. Estas propiedades se llaman **restricciones estructurales**. A continuación veremos la restricción estructural más importante para una relación que son la restricción de cardinalidad.

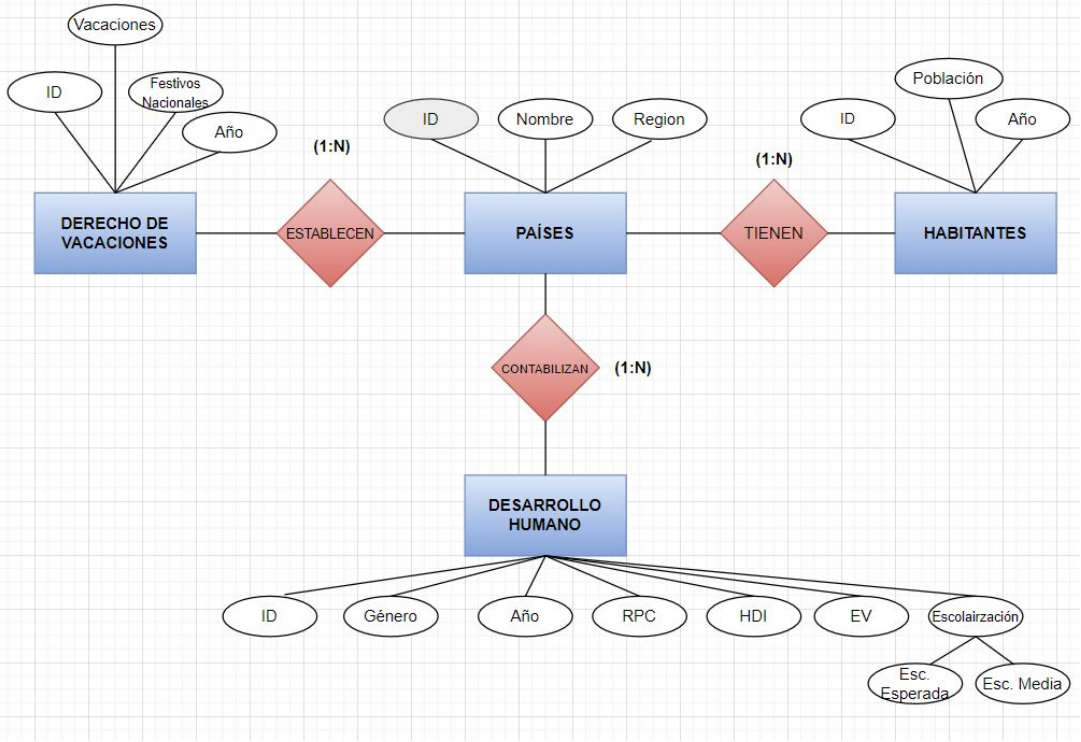
Cada relación entre entidades tiene una **cardinalidad**. La cardinalidad especifica el **número máximo de instancias de relación** en las que participa una entidad. Las restricciones de cardinalidad pueden ser:

a) **De uno a muchos (1:N)**, donde un elemento de la entidad A se relaciona con muchos elementos de la entidad B, pero cada elemento de la entidad B solo se relaciona con un elemento de la entidad A. Por ejemplo, en una ciudad pueden nacer muchos estudiantes pero un estudiante sólo puede nacer en una ciudad. Nótese que esta relación es equivalente a una relación de muchos a uno invirtiendo el orden de las entidades. La forma de representar esta restricción de cardinalidad en el modelo ER es etiquetando los dos lados de la relación con 1 (uno) o N (muchos).

b) **De uno a uno (1:1)**, donde un elemento de la entidad A se relaciona con un solo elemento de la entidad B y viceversa. Por ejemplo, un empleado dirige un único departamento y un departamento es dirigido por un único empleado. La forma de representar esta restricción de cardinalidad en el modelo ER es etiquetando los dos lados de la relación con 1 (uno).

c) **De muchos a muchos (N:M)**, donde un elemento de la entidad A se puede relacionar con muchos elementos de la entidad B y un elemento de la entidad B se puede relacionar con muchos elementos de la entidad A. Por ejemplo, un estudiante se puede matricular de muchas asignaturas y cada asignatura tiene matriculados a muchos estudiantes. La forma de representar esta restricción de cardinalidad en el modelo ER es etiquetando los dos lados de la relación con N (muchos) y M (muchos).

2 Modelado de Bases de Datos: Diseño Conceptual



2 Modelado de Bases de Datos: Diseño Lógico

- **Enteros** (números sin decimales):
  - **TINYINT** (1 byte, -128 a 127):
    - Ejemplo: `edad TINYINT` (edad de una persona entre 0 y 120).
  - **SMALLINT** (2 bytes, -32,768 a 32,767):
    - Ejemplo: `cantidad SMALLINT` (cantidad de productos en una orden).
  - **INT** (4 bytes, -2,147,483,648 a 2,147,483,647):
    - Ejemplo: `poblacion INT` (población de una ciudad).
  - **BIGINT** (8 bytes, gran capacidad):
    - Ejemplo: `ventas_anuales BIGINT` (ventas anuales en una empresa internacional).
- **Números de coma flotante** (números con decimales variables):
  - **FLOAT** (precisión simple, 4 bytes):
    - Ejemplo: `temperatura FLOAT` (registro de temperatura ambiental).
  - **DOUBLE** (precisión doble, 8 bytes):
    - Ejemplo: `latitud DOUBLE` (coordenada geográfica con alta precisión).
- **Números decimales** (números con precisión fija):
  - **NUMBER** (precisión y escala ajustables):
    - Ejemplo: `salario NUMBER(12,2)` (almacena un número con hasta 12 dígitos en total, de los cuales 2 son decimales, útil para valores financieros más amplios y precisos).

2 Modelado de Bases de Datos: Tipo de Dato

- **Booleanos** (valores lógicos):
  - **BOOLEAN** (TRUE/FALSE):
    - Ejemplo: `activo BOOLEAN` (indica si un cliente está activo o inactivo).
- **Cadenas de texto**:
  - **STRING** (secuencia de caracteres sin longitud fija):
    - Ejemplo: `descripcion STRING` (descripción larga de un producto).
  - **VARCHAR** (longitud variable hasta un límite definido):
    - Ejemplo: `nombre VARCHAR(255)` (nombre de una persona o empresa, con un máximo de 255 caracteres).
- **Fechas y Tiempos**:
  - **DATE** (solo fecha, formato YYYY-MM-DD):
    - Ejemplo: `fecha_nacimiento DATE` (fecha de nacimiento de una persona).
  - **TIMESTAMP** (fecha y hora con fracciones de segundo, formato YYYY-MM-DD HH:MM.s):
    - Ejemplo: `fecha_creacion TIMESTAMP` (momento exacto de la creación de un registro, incluyendo segundos fraccionados).
  - **DATETIME** (fecha y hora sin dependencia de zona horaria):
    - Ejemplo: `evento_inicio DATETIME` (hora de inicio de un evento, sin ajustarse a husos horarios).

2 Modelado de Bases de Datos: Diseño Lógico

Normalización en el Diseño Lógico

La normalización organiza los datos para evitar redundancias. Se sigue un proceso de cinco pasos, cada uno con reglas más estrictas:

- **1FN**: Todos los datos deben ser indivisibles. Por ejemplo, en lugar de guardar varios números de teléfono en una misma celda, se separan en filas diferentes.
- **2FN**: Cada columna debe depender completamente de la clave primaria. Si hay información que no depende del identificador principal, debe ir en otra tabla.
- **3FN**: Las columnas no clave no deben depender unas de otras. Solo deben depender de la clave primaria, evitando relaciones innecesarias entre los datos.
- **4FN**: Elimina las dependencias multivaluadas, es decir, aquellas en las que una celda tiene múltiples valores relacionados, como un profesor enseñando múltiples cursos. Se usa una tabla intermedia para resolver esto.
- **5FN**: Asegura que no haya relaciones que puedan descomponerse más sin perder información. Solo se mantiene lo necesario para que la estructura sea coherente.

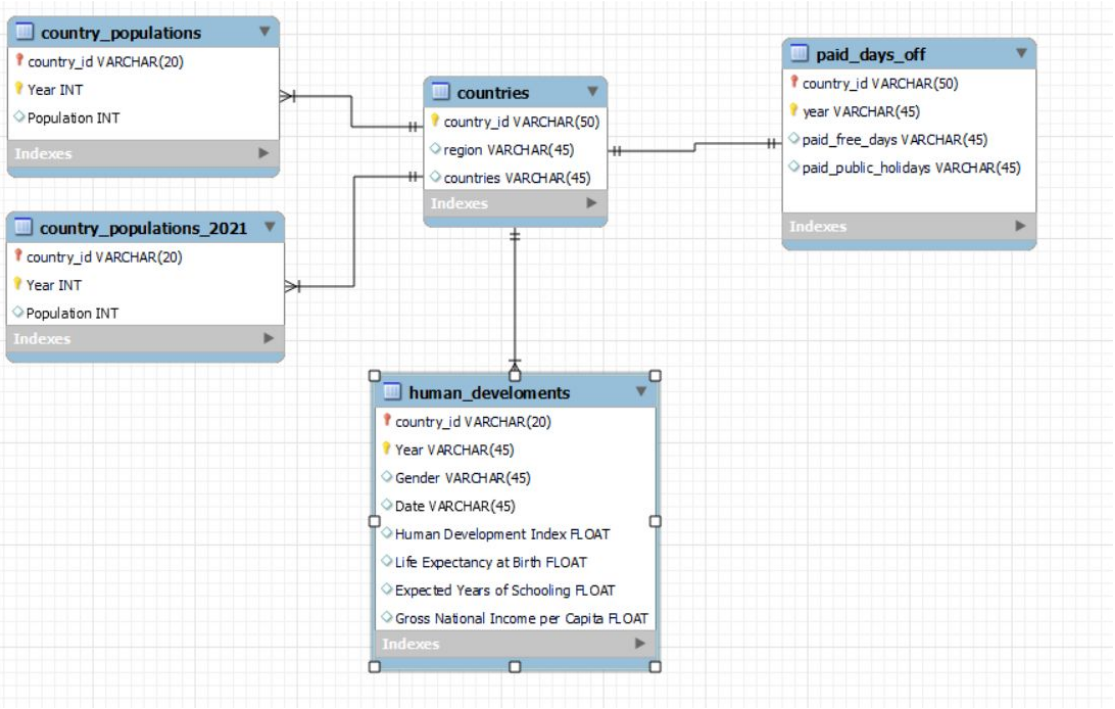
Beneficios:

- Facilita la organización de datos
- Mejora la eficiencia de consultas
- Evita duplicados

2 Modelado de Bases de Datos: Diseño Lógico

Traducción del Modelo Conceptual al Modelo Lógico

- **Objetivo**: Convertir el **modelo conceptual** en un **modelo lógico**.
  - Dependiente del tipo de **SGBD**: **Jerárquico**, **En red**, **Relacional** u **Orientado a Objetos**.
- **Especificaciones**:
  - El modelo lógico define las **estructuras de datos**: **Tablas**, **Objetos**, etc.
  - Es independiente del **SGBD específico** utilizado para la implementación.
- **Modelo Relacional**:
  - Detalles sobre la traducción del **Modelo Entidad-Relación** al **Modelo Relacional** en los anexos.



2 Modelado de Bases de Datos: Diseño Físico

Traducción del Modelo Lógico al Modelo Físico

- **Objetivo**: Convertir el **modelo lógico** en un **modelo físico** específico de un SGBD.
- **Estructuras Físicas**: Se implementan utilizando el **Lenguaje de Definición de Datos (DDL)** del SGBD.
- **Bases de Datos Relacionales**: Usan **SQL** para definir el modelo físico

```
create or replace TABLE HUMAN_DEVELOPMENTS (
    COUNTRY_ID VARCHAR(255),
    GENDER VARCHAR(255),
    YEAR NUMBER(38,0),
    HUMAN_DEVELOPMENT_INDEX NUMBER(38,2),
    LIFE_EXPECTANCY_AT_BIRTH NUMBER(38,2),
    EXPECTED_YEARS_OF_SCHOOLING NUMBER(38,2),
    MEAN_YEARS_OF_SCHOOLING NUMBER(38,2),
    GROSS_NATIONAL_INCOME_PER_CAPITA NUMBER(38,2)
);

create or replace TABLE COUNTRY_POPULATIONS (
    COUNTRY_ID VARCHAR(255),
    YEAR NUMBER(38,0),
    POPULATION NUMBER(38,0)
);
```

ntic master

UNIVERSIDAD COMPLUTENSE MADRID

3 Gestión de Bases de Datos

Este capítulo trata sobre los Sistemas de Gestión de Bases de Datos Relacionales (SGBDR), sus principales funciones como el almacenamiento, recuperación de datos, seguridad y control de acceso, así como la gestión de transacciones y concurrencia. También se introduce la plataforma de Data Warehouse en la nube Snowflake y sus características.

ntic master

UNIVERSIDAD COMPLUTENSE MADRID

3. Unit 3. Gestión de Bases de Datos

3.1. SGBD - Conceptos Generales

3.2. Plataforma Snowflake

3.3. Gestión de Usuarios

3.4. Interfaz de Usuario

3 Gestión de Bases de Datos: SGBDR

Sistemas de Gestión de Bases de Datos Relacionales (SGBDR)

- **Definición**:
  - Aplicaciones que permiten **almacenar, administrar, recuperar, modificar** y **manipular** datos en bases de datos relacionales.
- **Ejemplos populares**:
  - **Microsoft SQL Server**, **DB2**, **Oracle**, **MySQL**.
  - Permiten interactuar con la información almacenada mediante consultas y otras operaciones.
- **Relación con SQL**:
  - No todos los SGBDR usan SQL, pero la mayoría lo implementan y **extienden**.
  - Ejemplos de extensiones:
    - **Transact SQL (T-SQL)** en Microsoft SQL Server.
    - **PL/SQL** en Oracle.



3 Gestión de Bases de Datos: Snowflake

Snowflake: Plataforma de Data Warehouse en la Nube

- **¿Qué es Snowflake?**
  - Plataforma de **data warehouse** en la nube, diseñada para el **almacenamiento** y **análisis escalable** de datos.
- **No es un SGBD tradicional**
  - A diferencia de sistemas como **MySQL** o **SQL Server**, Snowflake está enfocado en el manejo de grandes volúmenes de datos, no en transacciones.
- **Almacenamiento y análisis**
  - Optimizado para **almacenar, consultar y analizar** grandes conjuntos de datos, integrando **almacenamiento y procesamiento** en la nube.

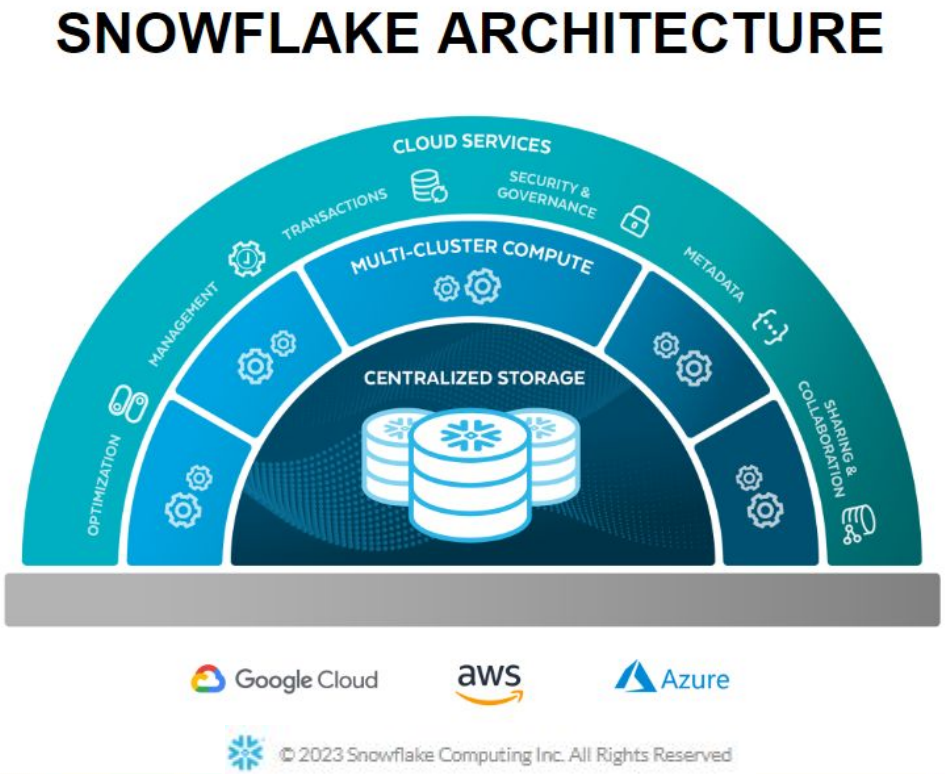


3 Gestión de Bases de Datos: Snowflake

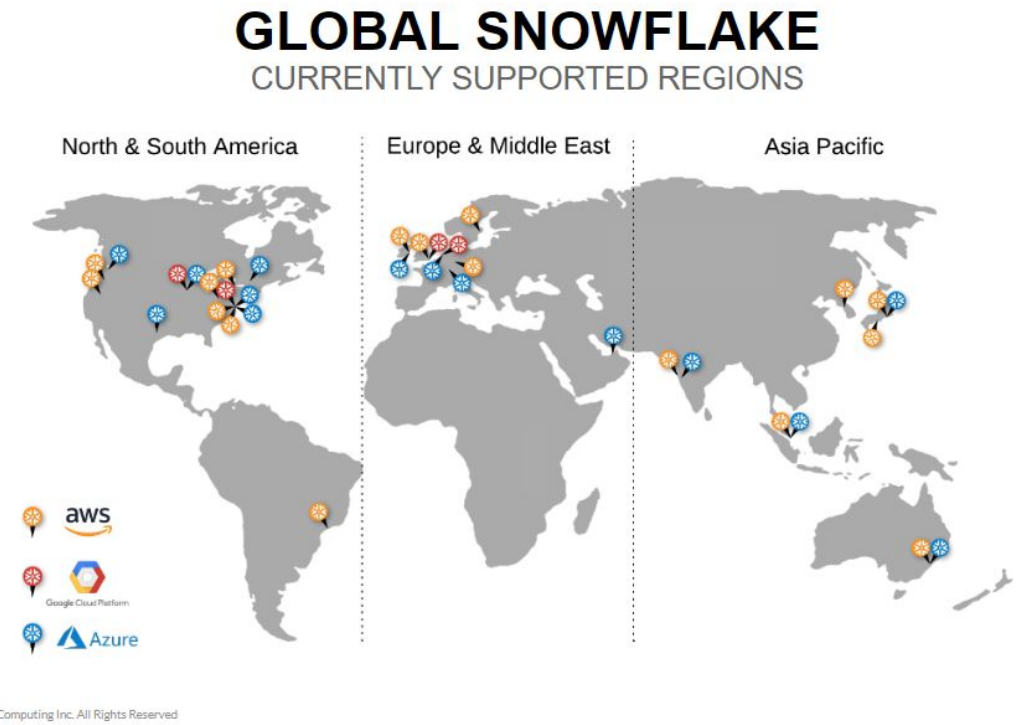
En el video a continuación, podemos ver algunos aspectos diferenciadores de Snowflake los cuales lo convierten en uno de los proveedores cloud data warehouse más potentes del mercado



3 Gestión de Bases de Datos: Snowflake



3 Gestión de Bases de Datos: Snowflake

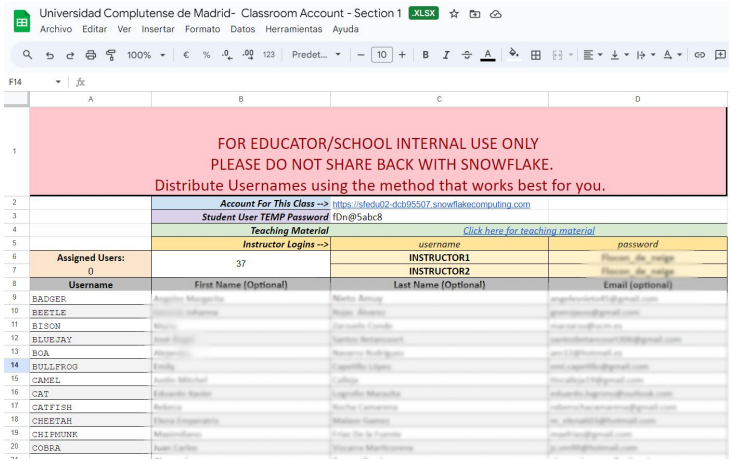


3 Gestión de Bases de Datos: Snowflake

Vamos a trabajar en los laboratorios de **Snowflake for Academia** habilitados exclusivamente para el Máster

1

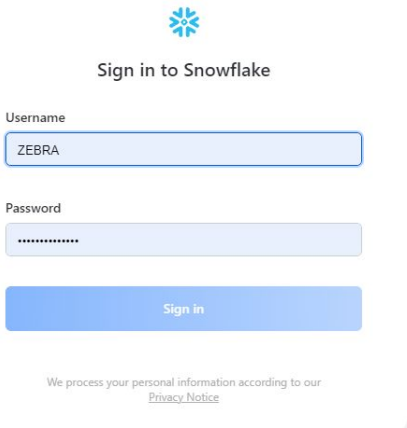
Busca tu nombre y el usuario asignado en la hoja de cálculo que el profesor compartirá en el foro. Es muy importante que uses solo tu USERNAME pues este es <u>personal e intransferible</u>



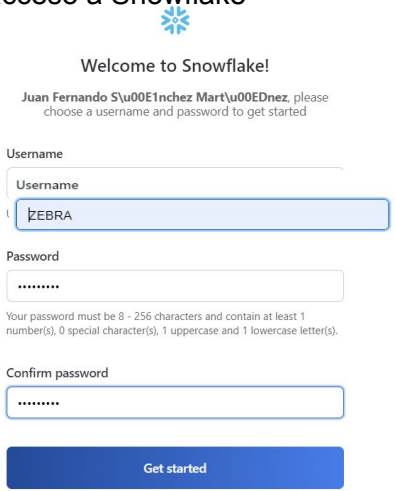
3 Gestión de Bases de Datos: Snowflake

Ahora vamos a nuestra bandeja de entrada, abrimos el email que acabamos de recibir y procedemos a activar la cuenta. Hecho esto nos pedirá activar la cuenta y crear un usuario:

2 **Introduce tu Username (el nombre asignado en la hoja de cálculo)** e introduce la contraseña genérica

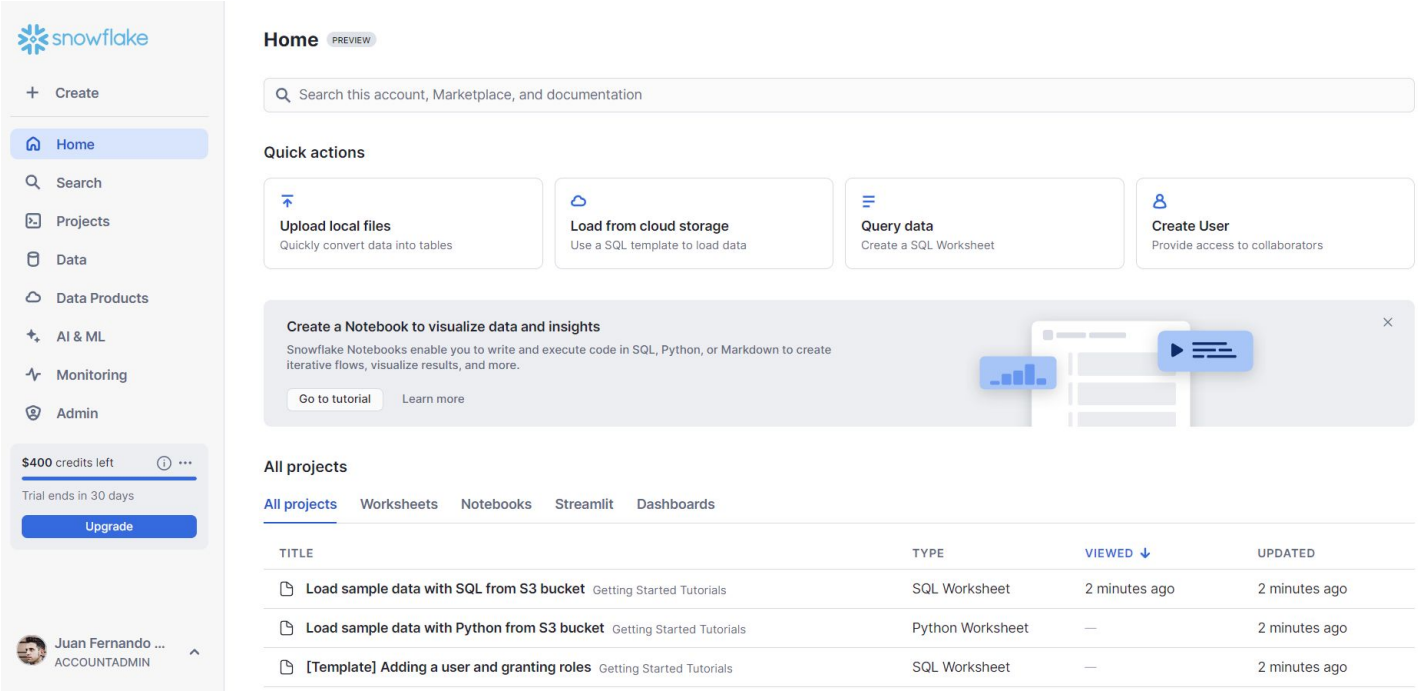


3 **A continuación introducimos una nueva contraseña.** Es importante que la guardes pues es tu vía de acceso a Snowflake



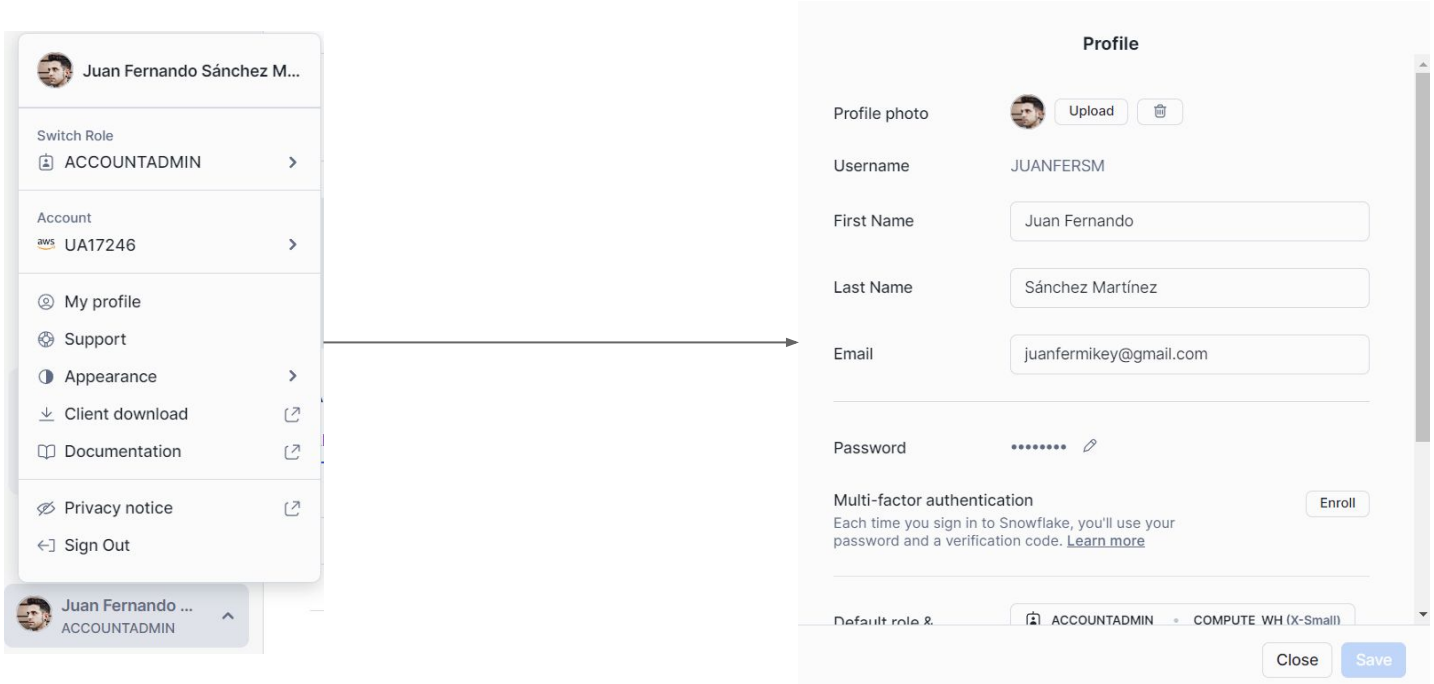
3 Gestión de Bases de Datos: Snowflake

De esta manera habremos accedido a **nuestra consola de Snowflake** y veremos lo siguiente:



3 Gestión de Bases de Datos: Snowflake

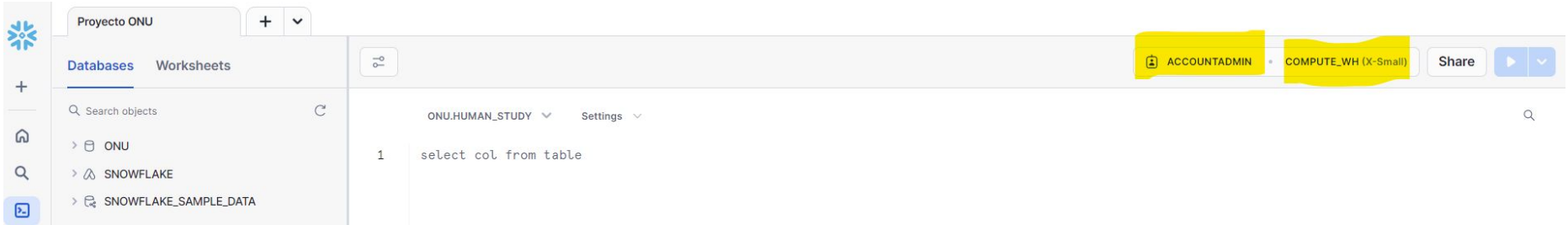
A continuación, vamos ajustar nuestro perfil para ajustarlo a nuestras preferencias:



3 Gestión de Bases de Datos: Snowflake

Conceptos Clave en Snowflake: Roles y Warehouses

- **Roles**:
  - Los **roles** controlan el acceso a objetos en Snowflake mediante **privilegios**.
  - A un usuario se le pueden asignar múltiples roles, permitiendo cambiar entre ellos según la acción a realizar.
  - Cada rol define qué funciones comerciales o acciones puede ejecutar un usuario sobre los datos.
- **Warehouses**:
  - Un **virtual warehouse** es un grupo de **recursos computacionales** asignados para procesar consultas en Snowflake.
  - Define cuántos recursos se destinan para ejecutar queries, afectando la **velocidad** de procesamiento.
- **Importancia**:
  - Los **roles** y **warehouses** determinan la **seguridad** y el **rendimiento** de las consultas.
  - Es crucial seleccionar correctamente ambos elementos antes de ejecutar cualquier consulta.



Acceso y Gestión en Snowflake: Bases de Datos y Worksheets

- **Bases de Datos**:
  - En esta sección, puedes acceder a todas tus bases de datos en Snowflake.
  - **Schemas**: Agrupaciones lógicas de objetos en la base de datos (tablas, vistas, procedimientos almacenados).
  - Desde aquí es posible interactuar con los principales componentes de la base de datos, como:
    - **Tablas**: Estructuras que almacenan los datos de manera organizada en filas y columnas.
    - **Vistas**: Tablas virtuales que resultan de consultas predefinidas, facilitando el acceso a datos filtrados o combinados.
    - **Procedimientos Almacenados**: Código SQL que se ejecuta dentro de la base de datos para automatizar tareas repetitivas.
  - Puedes consultar, modificar y gestionar estos objetos directamente desde la interfaz

3 Gestión de Bases de Datos: Snowflake

EJERCICIO GUIADO

1 Introduction to Snowflake and Snowflake Objects

The purpose of this lab is to familiarize you with Snowflake's Snowsight user interface. Specifically, you will learn how to use Snowsight worksheets to create and use Snowflake objects that you will use to run queries and in your day-to-day work.

Learning Objectives:

- How to navigate Snowsight to find the tools you'll need
- How to create and manage folders and worksheets
- How to set the context
- How to create warehouses, databases, schemas, and tables
- How to run a simple query

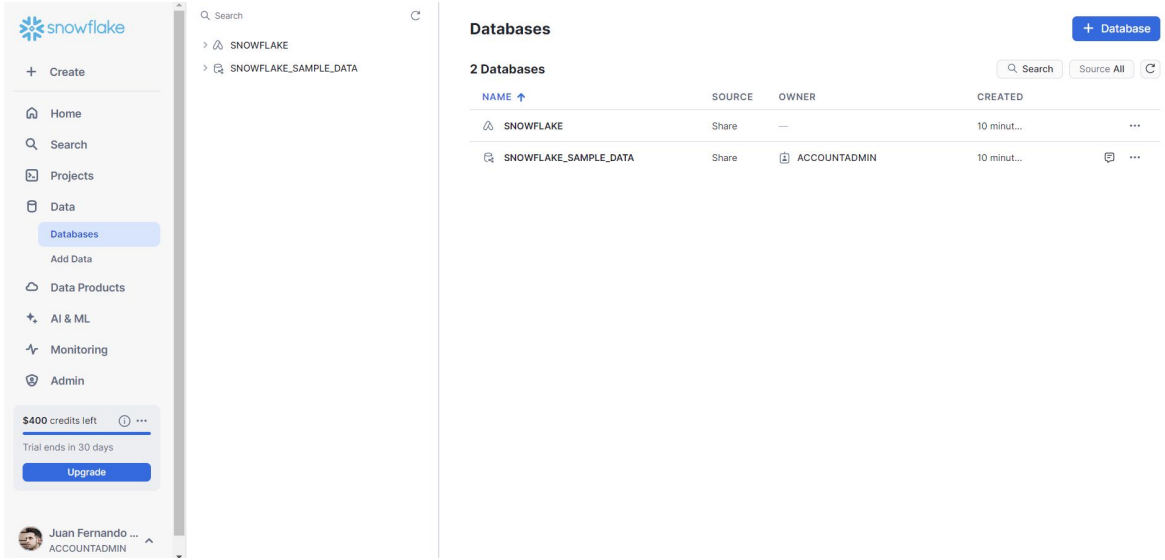
Scenario:

You are working for Snowbear Air, which is an airline that flies to fun destinations all over the world. You've been asked to create a few Snowflake objects in a development environment to test out your SQL statements. You will need to:

- Create a database and a schema
- Create a virtual warehouse
- Create a table and populate it with data
- Run some test queries

3 Gestión de Bases de Datos: Snowflake

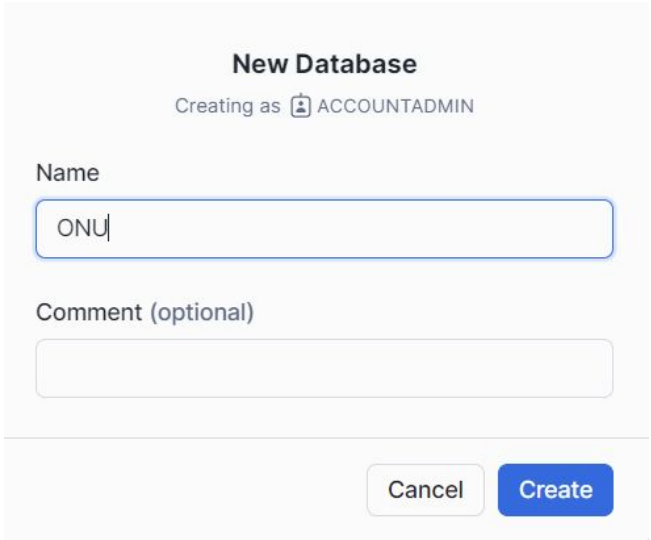
Data en Snowflake: La sección **Data** en Snowflake permite acceder y gestionar las **bases de datos**, **tablas**, **esquemas**, y **vistas**. Aquí puedes explorar los datos, cargar nuevos conjuntos, y administrar los objetos de tu base de datos. Además, puedes realizar operaciones como crear, modificar, o eliminar tablas y visualizar sus contenidos de manera directa.



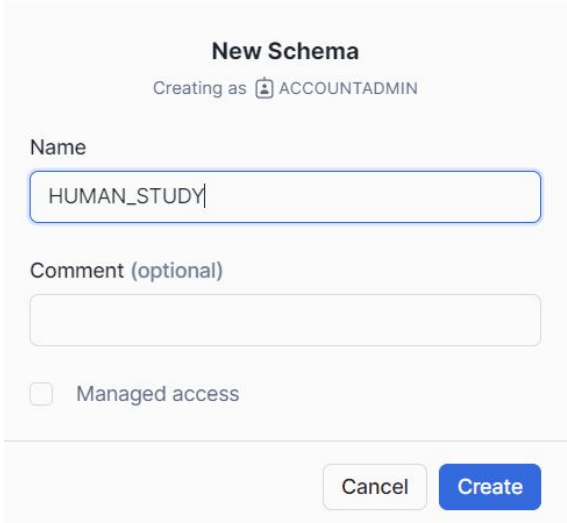
3 Gestión de Bases de Datos: Snowflake

Creación de objetos en Snowflake

- DataBase



- Schema

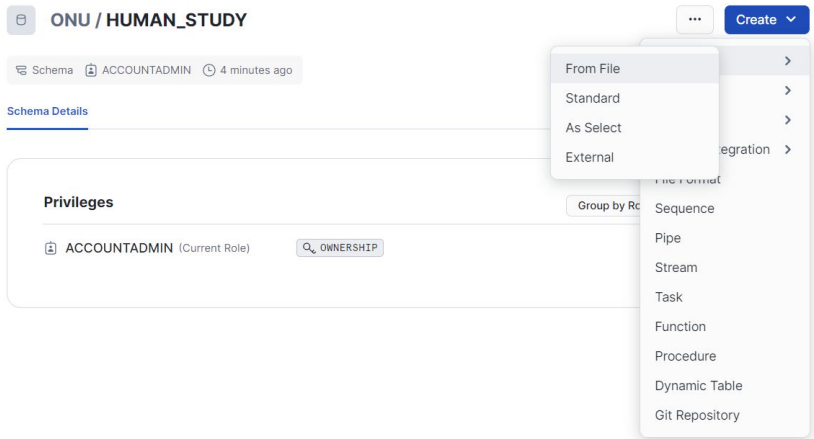


3 Gestión de Bases de Datos: Snowflake

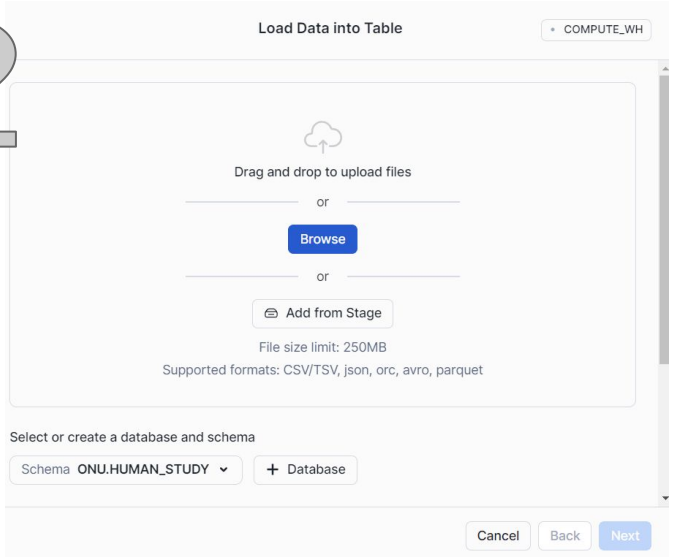
Creación de objetos en Snowflake

- Table: A continuación, vamos a **cargar datos dentro de nuestra tabla**. Para ello, descargamos del portal de master los CSV y seleccionamos la opción “Load Table” de la parte superior derecha.

1

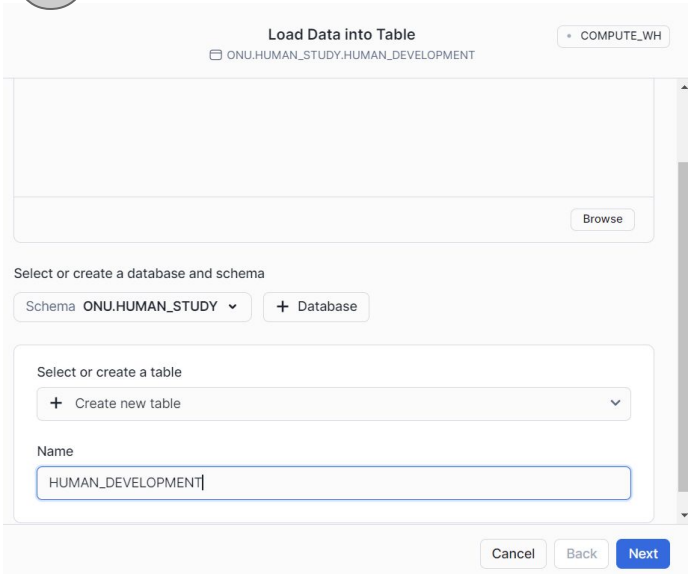


2

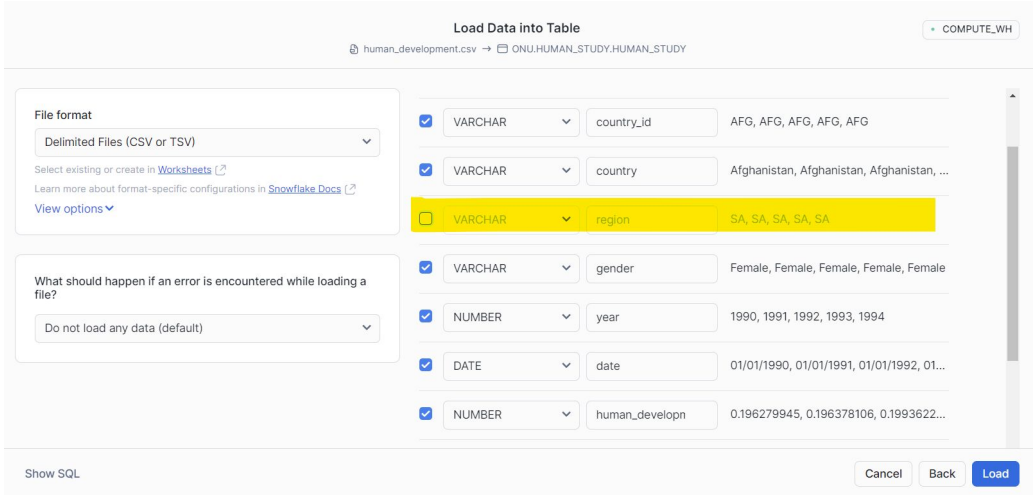


3 Gestión de Bases de Datos: Snowflake

3



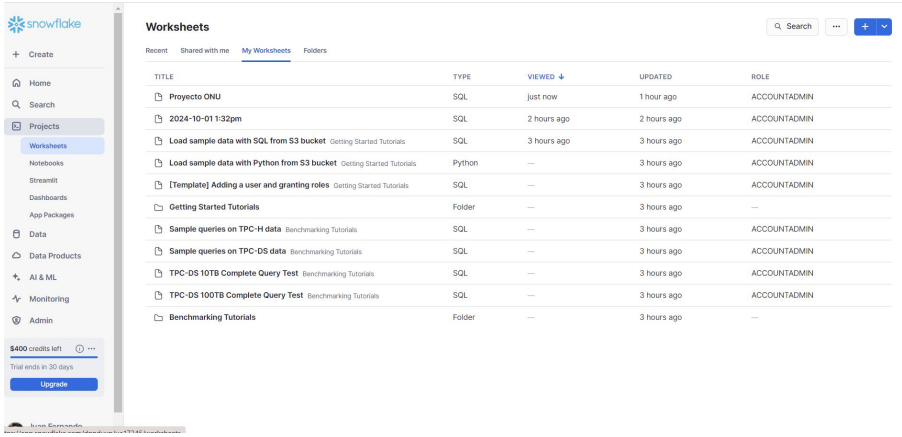
4



3 Gestión de Bases de Datos: Snowflake

Worksheets:

- Las **Worksheets** son las áreas de trabajo donde puedes ejecutar consultas SQL y manipular los datos almacenados
- La interfaz está dividida en dos partes:
  - **Izquierda**: Acceso rápido a tus bases de datos, tablas y vistas, permitiendo una navegación sencilla y eficiente.
  - **Derecha**: Consola de consultas donde puedes escribir y ejecutar **queries SQL**.
- Las **Worksheets** permiten realizar diversas operaciones, como:
  - Ejecutar consultas **SELECT** para extraer información.
  - Modificar datos con **INSERT**, **UPDATE** o **DELETE**.
  - Crear nuevas tablas o vistas con sentencias **CREATE**.
  - Realizar análisis complejos usando las capacidades de Snowflake.



3 Gestión de Bases de Datos: Snowflake

- **¿Qué es el contexto en Snowflake?**
  - El **contexto** en Snowflake define el entorno en el que se ejecutan las consultas y operaciones, especificando qué **base de datos**, **esquema**, **warehouse** y **rol** están activos.
- **Componentes clave del contexto**:
  - **Base de Datos**: El conjunto de datos con los que se trabaja.
  - **Esquema**: La estructura organizativa dentro de la base de datos que agrupa tablas, vistas y otros objetos.
  - **Warehouse**: El recurso computacional que se utilizará para procesar las consultas.
  - **Rol**: Define los **permisos** y accesos que tiene el usuario para interactuar con los objetos y ejecutar consultas.
    i. Diferentes roles otorgan diferentes niveles de acceso, desde lectura hasta manipulación completa de datos.
- **Importancia del contexto**:
  - El contexto asegura que las consultas SQL interactúan con los datos y recursos correctos.
  - **Warehouse**: Afecta el **rendimiento** de las consultas.
  - **Rol**: Asegura que el usuario tiene los permisos adecuados para realizar las operaciones.
  - **Base de Datos y Esquema**: Garantizan que las operaciones ocurren en el conjunto de datos correcto.
- **Gestión del contexto**:
  - Puedes cambiar el contexto mediante comandos SQL como:
    i. `USE ROLE nombre_rol;`
    ii. `USE DATABASE nombre_base_datos;`
    iii. `USE SCHEMA nombre_esquema;`
    iv. `USE WAREHOUSE nombre_warehouse;`

ntic master

UNIVERSIDAD COMPLUTENSE MADRID



Lenguaje SQL

En esta sección se exploran los fundamentos para trabajar con bases de datos, combinando gestión estructural y manipulación de datos. Se aborda la gestión estructural mediante comandos DDL como CREATE, ALTER, DROP y UNDROP para administrar objetos del sistema, así como el dominio del Lenguaje de Manipulación de Datos desde consultas básicas con SELECT, WHERE, ORDER BY y TOP hasta técnicas avanzadas que incluyen agrupación, combinación de tablas, subconsultas, vistas, funciones de ventana y CTEs

ntic master

UNIVERSIDAD COMPLUTENSE MADRID

4. Unit 4. Lenguaje SQL

4.1. DDL (Data Definition Language)

4.1.1. CREATE - Creación de Estructuras

4.1.2. ALTER - Modificación de Estructuras

4.1.3. DROP - Eliminación de Objetos

4 Lenguaje DDL

DDL en Snowflake: CREATE, DROP, ALTER y UNDROP

En Snowflake, el **Lenguaje de Definición de Datos (DDL)** se utiliza para gestionar la estructura de la base de datos. Los comandos más comunes son:

- **CREATE**: Crea nuevos objetos en la base de datos, como tablas, vistas o esquemas.
  Ejemplo: `CREATE TABLE clientes (id INT, nombre VARCHAR(50));`
- **DROP**: Elimina objetos existentes, como tablas o vistas.
- **ALTER**: Modifica la estructura de un objeto existente. Se puede utilizar, por ejemplo, para **cambiar el tipo de dato** de una columna.
  Ejemplo: `ALTER TABLE clientes ALTER COLUMN edad SET DATA TYPE VARCHAR(3);`
  (Cambia el tipo de dato de la columna "edad" de **INT** a **VARCHAR(3)**).
  Ejemplo: `DROP TABLE clientes;` (Elimina la tabla "clientes").
- **UNDROP**: Recupera un objeto eliminado previamente con **DROP**, siempre que esté dentro del período de retención de datos.
  Ejemplo: `UNDROP TABLE clientes;` (Restaura la tabla "clientes" que fue eliminada).

4 Lenguaje DDL

CREATE: La instrucción **CREATE** se utiliza para crear nuevos objetos en la base de datos, como tablas, vistas o índices.

1

```
CREATE DATABASE IF NOT EXISTS ONU;
```

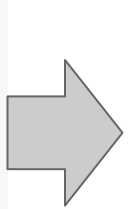
2

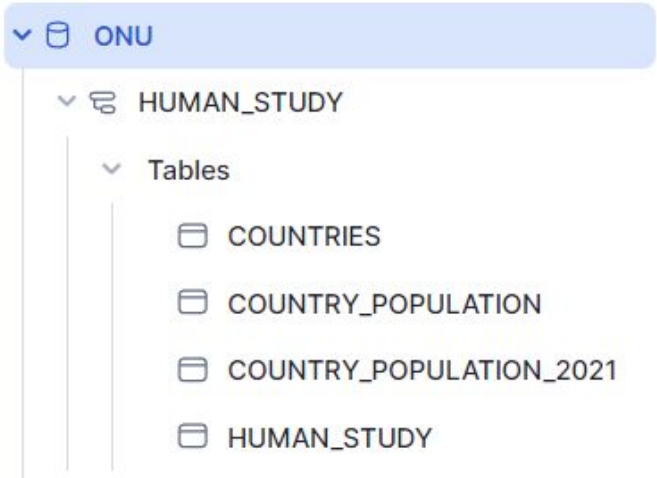
```
CREATE SCHEMA IF NOT EXISTS  "ONU"."HUMAN_STUDY";
```

3

```
    COUNTRY_ID VARCHAR(16777216),
    COUNTRY VARCHAR(16777216),
    GENDER VARCHAR(16777216),
    YEAR NUMBER(38,0),
    DATE DATE,
    HUMAN_DEVELOPMENT_INDEX NUMBER(38,9),
    LIFE_EXPECTANCY_AT_BIRTH NUMBER(38,4),
    EXPECTED_YEARS_OF_SCHOOLING NUMBER(38,9),
    MEAN_YEARS_OF_SCHOOLING NUMBER(38,9),
    GROSS_NATIONAL_INCOME_PER_CAPITA NUMBER(38,7)
);
```

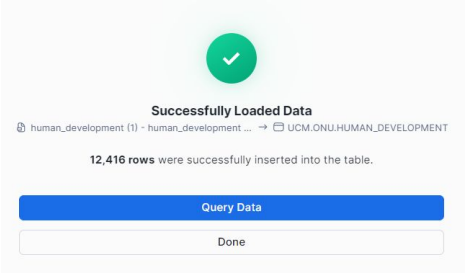
Show less



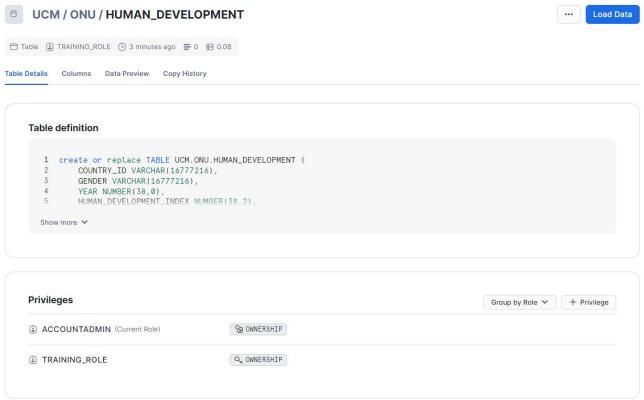


4 Lenguaje DDL

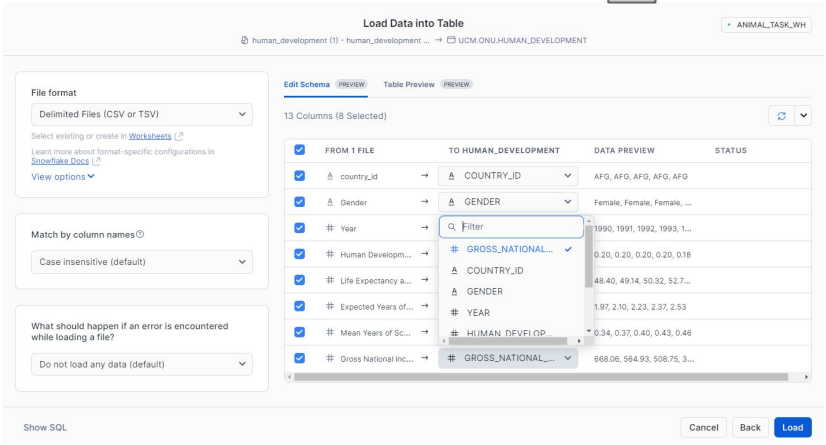
CREATE: Se utiliza para crear una nueva tabla en la base de datos, definiendo el nombre de la tabla y las columnas con sus respectivos tipos de datos.



4



5



4 Lenguaje DDL

ALTER: La instrucción **ALTER** se utiliza para modificar la estructura de una tabla ya existente, como añadir o eliminar columnas, o cambiar tipos de datos.

ONU.HUMAN_STUDY Settings

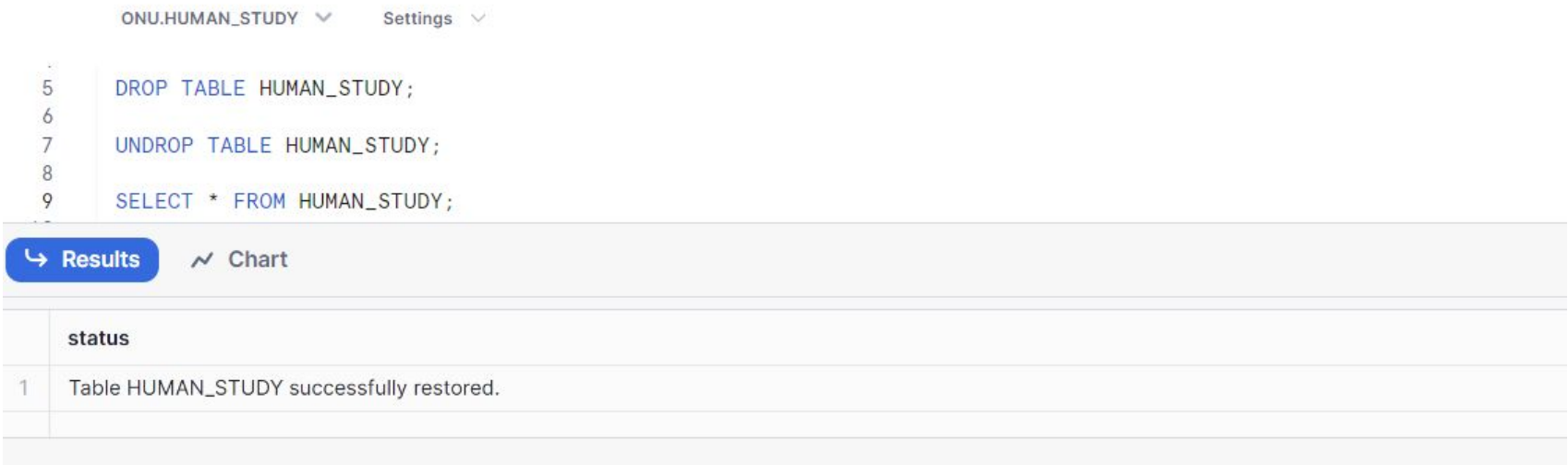
```
1  ALTER TABLE ONU.HUMAN_STUDY.HUMAN_STUDY RENAME COLUMN GROSS_NATIONAL_INCOME_PER_CAPITA TO RPC;
2
3  DESCRIBE TABLE HUMAN_STUDY;
4
5
```

Results Chart

	name	type	kind	null?	default	primary key	unique key	check	expression	comment
1	COUNTRY_ID	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null
2	COUNTRY	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null
3	GENDER	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null
4	YEAR	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null	null
5	DATE	DATE	COLUMN	Y	null	N	N	null	null	null
6	HUMAN_DEVELOPMENT_INDEX	NUMBER(38,9)	COLUMN	Y	null	N	N	null	null	null
7	LIFE_EXPECTANCY_AT_BIRTH	NUMBER(38,4)	COLUMN	Y	null	N	N	null	null	null
8	EXPECTED_YEARS_OF_SCHOOLING	NUMBER(38,9)	COLUMN	Y	null	N	N	null	null	null
9	MEAN_YEARS_OF_SCHOOLING	NUMBER(38,9)	COLUMN	Y	null	N	N	null	null	null
10	RPC	NUMBER(38,7)	COLUMN	Y	null	N	N	null	null	null

4 Lenguaje DDL

DROP : Se usa para eliminar una tabla completa de la base de datos. Es importante usar este comando con precaución, ya que una vez que una tabla es eliminada, todos los datos en ella se pierden permanentemente, a menos que existan mecanismos de respaldo o recuperación como en sistemas de bases de datos avanzados como Snowflake.



UNDROP: UNDROP TABLE es una característica de Snowflake que permite restaurar tablas que han sido eliminadas recientemente. Esta operación es muy útil en casos donde una eliminación fue accidental o prematura.

ntic master

UNIVERSIDAD COMPLUTENSE MADRID

4. Unit 4. Lenguaje SQL

DML (Data Manipulation Language)

- **Consultas Básicas**

■UPDATE/DELETE - Modificación

■SELECT - Selección de Datos

■DISTINCT/AS - Alias y Únicos

■WHERE - Filtrado

■ORDER BY - Ordenación

■Funciones: Escalares - Agregación

■GROUP BY - Agrupación

4 Lenguaje DML: Queries I

En las sesiones se trabajará el ejemplo del Proyecto ONU el cual se puede encontrar en la documentación del Campus Virtual.



4 Lenguaje DML: Queries Básicas

UPDATE y DELETE

La instrucción **UPDATE** se utiliza para modificar los valores existentes en una o más filas de una tabla. A través de la cláusula **SET**, se especifica la columna que debe actualizarse y su nuevo valor. La cláusula **WHERE** indica las filas específicas que deben ser modificadas. Sin la cláusula **WHERE**, todas las filas de la tabla serán actualizadas.

```
UPDATE tabla
SET columna1 = 'nuevo_valor'
WHERE columna2 = 'condicion';
```

DELETE en SQL se utiliza para eliminar registros específicos de una tabla. A diferencia de **DROP**, que elimina una tabla completa, **DELETE** solo elimina las filas que cumplen con una condición particular. Es importante utilizar el **WHERE** para evitar eliminar todos los registros de una tabla por error.

4 Lenguaje DML: Queries Básicas

SELECT

- **SELECT** es una de las instrucciones más utilizadas en SQL para **recuperar datos** de la base de datos.
  - Permite formar consultas que devuelven los datos requeridos.
- **Sintaxis Básica**:
  - La instrucción SELECT se compone de varias **cláusulas**. Las únicas **requeridas** son **SELECT** y **FROM**.
  - Cláusulas opcionales que refinan la consulta:
    1. **WHERE**: Filtra filas basadas en una condición.
    2. **GROUP BY**: Agrupa las filas que tienen valores en común.
    3. **HAVING**: Filtra grupos de datos, a menudo con funciones de agregación.
- **Sintáxis Básica**:



4 Lenguaje DML: Queries Básicas

SELECT y FROM

La lista de columnas de la cláusula **SELECT** puede contener los siguientes elementos:

- El símbolo asterisco (*), que significa que queremos seleccionar todas las columnas de una tabla.
- Una lista de columnas.
- Nombre_columna AS nombre, donde nombre es un alias de Nombre_columna y reemplaza al nombre de la columna en el resultado de la consulta.
- Una expresión.
- Una función de agregación: MIN, MAX, AVG, COUNT, SUM.

Por ejemplo, considere la consulta siguiente:

```
-- SELECT
SELECT columna1, columna2
FROM tabla;
```

4 Lenguaje DML: Queries Básicas

ALIAS

Los **alias** permiten asignar nombres temporales a columnas o tablas en una consulta, haciendo que los resultados sean más legibles y fáciles de interpretar.

- **Alias para Columnas**: Se utiliza para renombrar columnas en los resultados.
- **Alias para Tablas**: Se utiliza para acortar o simplificar el nombre de una tabla dentro de la consulta.

Los **alias** son útiles para mejorar la claridad de las consultas y cuando se trabaja con uniones o funciones complejas.

```sql
-- ALIAS para Columnas
SELECT columna1 AS alias_columna1,
       columna2 AS alias_columna2
FROM tabla;
```

```sql
-- ALIAS para Tablas
SELECT t.columna1, t.columna2
FROM tabla AS t;
```

4 Lenguaje DML: Queries Básicas

SELECT y FROM

DISTINCT:

- Elimina **filas duplicadas** de los resultados de la consulta.

```
-- DISTINCT
SELECT DISTINCT columna1
FROM tabla;
```

ALL:

- Devuelve **todas** las filas, incluidas las duplicadas.
- **ALL** es el comportamiento por defecto si no se especifica ninguna de las dos.

```
-- SELECT ALL
SELECT ALL columna1, columna2
FROM tabla;
```

4 Lenguaje DML: Queries Básicas

WHERE

- **Función**: Filtra las filas devueltas por **FROM** según una condición de búsqueda.

Operadores de Comparación:

- = Igual
- <> o != Distinto
- < Menor que
- > Mayor que
- <= Menor o igual que
- >= Mayor o igual que
- !> No mayor que
- !< No menor que

```sql
--OPERADORES DE COMPARACIÓN
-- Ejemplo con = (igual a)
SELECT columna1, columna2
FROM tabla
WHERE columna1 = 'valor';

-- Ejemplo con <> (distinto de)
SELECT columna1, columna2
FROM tabla
WHERE columna1 <> 'valor';

-- Ejemplo con > (mayor que)
SELECT columna1, columna2
FROM tabla
WHERE columna1 > 100;

-- Ejemplo con < (menor que)
SELECT columna1, columna2
FROM tabla
WHERE columna1 < 100;

-- Ejemplo con >= (mayor o igual que)
SELECT columna1, columna2
FROM tabla
WHERE columna1 >= 50;

-- Ejemplo con <= (menor o igual que)
SELECT columna1, columna2
FROM tabla
WHERE columna1 <= 50;
```

4 Lenguaje DML: Queries Básicas

Operadores Lógicos

Los operadores lógicos se utilizan en la cláusula **WHERE** para conectar múltiples condiciones:

- **AND**: La consulta solo devuelve resultados si **ambas condiciones** se cumplen.
- **OR**: Devuelve las filas que cumplan **al menos una** de las condiciones.
- **NOT**: Invierte el valor lógico de la condición, es decir, si la condición es `TRUE`, se convierte en `FALSE`, y viceversa.

```sql
-- Operadores Lógicos: AND, OR, NOT
SELECT *
FROM tabla
WHERE columna1 > 100
AND columna2 = 'valor';

SELECT *
FROM tabla
WHERE columna1 > 100
OR columna2 = 'valor';

SELECT *
FROM tabla
WHERE NOT columna1 > 100;
```

4 Lenguaje DML: Queries Básicas

Operadores IN y BETWEEN

- **IN**: Especifica una lista de valores. Devuelve TRUE si el valor de la columna coincide con **alguno** de los valores listados.

```
-- IN
SELECT *
FROM tabla
WHERE columna1 IN ('valor1', 'valor2', 'valor3');
```

- **BETWEEN**: Especifica un **rango de valores**. Devuelve TRUE si el valor de la columna está dentro del rango, inclusive de los límites.

```
-- BETWEEN
--Números
SELECT *
FROM tabla
WHERE columna1 BETWEEN 100 AND 200;

-- Fechas
SELECT columna1, columna_fecha
FROM tabla
WHERE columna_fecha BETWEEN '2023-01-01' AND '2023-12-31';
```

NULL

```
-- NULL
SELECT *
FROM tabla
WHERE columna1 IS NULL;
```

- puede permitir en columnas utilizando la palabra clave **NULL** al crear la tabla.**NULL**: Representa un valor **desconocido** o **no aplicable**. Se
- **Comparaciones con NULL**: Las comparaciones con valores **NULL** se evalúan como **FALSAS**.
- Para buscar valores **NULL**, se utiliza **IS NULL** o **IS NOT NULL**:

4 Lenguaje DML: Queries Básicas

Cláusula ORDER BY en SQL

La cláusula **ORDER BY** se utiliza para **ordenar** los resultados de una consulta en base a una o más columnas. Puede ordenar en forma **ascendente** o **descendente**.

Opciones de Ordenación:

- **ASC**: Ordena en **sentido ascendente** (valor por defecto).
  - Ejemplo: De menor a mayor, de "A" a "Z".
- **DESC**: Ordena en **sentido descendente**.
  - Ejemplo: De mayor a menor, de "Z" a "A".

```
-- ORDER BY
SELECT columna1, columna2
FROM tabla
ORDER BY columna1 [ASC | DESC];
```

4 Lenguaje DML: Queries Básicas

FUNCIONES ESCALARES TEXTO

- **UPPER**:
  - Convierte todos los caracteres de una cadena de texto a **mayúsculas**.
- **LOWER**:
  - Convierte todos los caracteres de una cadena de texto a **minúsculas**.

```
-- FUNCIONES: UPPER, LOWER, CONCAT, TRUNCATE
SELECT UPPER(columna1), LOWER(columna2)
FROM tabla;
```

4 Lenguaje DML: Queries Básicas

FUNCIONES ESCALARES NUMÉRICAS

- **POWER(base, exponente)**:
  - Calcula la **potencia** de un número, elevando una base a un exponente.
  - Ejemplo: `POWER(2, 3)` devuelve `8` (2^3 = 8).
- **SQRT(número)**:
  - Devuelve la **raíz cuadrada** de un número.
  - Ejemplo: `SQRT(16)` devuelve `4`.
- **ABS(número)**:
  - Devuelve el **valor absoluto** de un número, es decir, su valor sin el signo.
  - Ejemplo: `ABS(-5)` devuelve `5`.

```
-- FUNCIONES MATEMÁTICAS: POWER, SQRT, ROUND
SELECT POWER(columna1, 2), SQRT(columna2), ROUND(columna3, 1)
FROM tabla;
```

4 Lenguaje DML: Queries Básicas

OTRAS FUNCIONES

- **CONCAT(cadena1, “separador”, cadena2, ...)**:
  - **Concatena** (une) dos o más cadenas de texto en una sola.
  - Ejemplo: `CONCAT('Hola, ', 'mundo')` devuelve `'Hola, mundo'`.
  - Puedes concatenar múltiples cadenas de forma secuencial para crear un solo resultado.
- **TRUNCATE (columna 1, 2):**
  - Trunca (corta) un número decimal eliminando los dígitos después del número de decimales especificado, sin realizar redondeo.
  - Ejemplo: TRUNCATE(3.14159, 2) devuelve 3.14 (corta en el segundo decimal sin redondear).
- **ROUND(número, decimales)**:
  - Redondea un número a la cantidad de **decimales** especificados.
  - Ejemplo: `ROUND(3.14159, 2)` devuelve `3.14`.

```
SELECT CONCAT(columna1, columna2)
FROM tabla;

SELECT TRUNCATE(columna1, 2)
FROM tabla;
```

4 Lenguaje DML: Queries Básicas

Manejo de los NULOS

En SQL, los **valores nulos (NULL)** indican la ausencia de un valor. Para trabajar con **NULL**, existen varias funciones que permiten detectar y reemplazar estos valores en las consultas.

Funciones para Manejar Nulos:

1. **IS NULL**: Verifica si una columna contiene un valor nulo.
2. **IS NOT NULL**: Verifica si una columna **no** contiene un valor nulo.
3. **IFNULL()**: Reemplaza los valores nulos con un valor predeterminado.
4. **COALESCE()**: Devuelve el primer valor no nulo de una lista de expresiones (utilizado en bases de datos como SQL Server y PostgreSQL).
5. **NULLIF()**: Devuelve `NULL` si dos valores son iguales, de lo contrario devuelve el primer valor.

```sql
-- Reemplazar NULL con un valor predeterminado (IFNULL)
SELECT IFNULL(columna1, 'Valor por defecto') AS columna_con_valor
FROM tabla;

-- Devolver el primer valor no nulo (COALESCE)
SELECT COALESCE(columna1, columna2, 'Valor por defecto') AS columna_no_nula
FROM tabla;

-- Comparar y devolver NULL si dos valores son iguales (NULLIF)
SELECT NULLIF(columna1, columna2) AS resultado
FROM tabla;
```

4 Lenguaje DML: Queries Básicas

Funciones de Agregación en SQL

Las **funciones de agregación** permiten realizar cálculos sobre un conjunto de valores y devolver un único resultado. Son esenciales para generar reportes y resumir grandes volúmenes de datos. Las funciones más comunes son:

- **COUNT()**: Cuenta el número de filas.
- **SUM()**: Calcula la suma de un conjunto de valores.
- **AVG()**: Calcula el promedio de los valores.
- **MIN()**: Devuelve el valor mínimo.
- **MAX()**: Devuelve el valor máximo.
- **STDEV():** Devuelve la Desviación típica
- **MEDIAN():** Devuelve la Mediana

```sql
--: COUNT, SUM, AVG, MAX, MIN
SELECT
    COUNT(columna1) AS total_filas,
    SUM(columna2) AS suma_total,
    AVG(columna2) AS promedio_valores,
    MAX(columna2) AS valor_maximo,
    MIN(columna2) AS valor_minimo
FROM tabla
WHERE condicion;
```

Estas funciones se suelen usar junto con la cláusula **GROUP BY** para agrupar datos y aplicar las funciones sobre cada grupo.

4 Lenguaje DML: Queries Básicas

GROUP BY

La cláusula **GROUP BY** se utiliza para agrupar filas que tienen los mismos valores en una o más columnas. Es comúnmente utilizada junto con **funciones de agregación** como **COUNT**, **SUM**, **AVG**, **MIN**, y **MAX** para realizar cálculos sobre cada grupo de datos en lugar de sobre todas las filas de la tabla.

- **Funcionamiento**:
  - Las filas de la tabla se agrupan de acuerdo a los valores de las columnas especificadas en **GROUP BY**.
  - Para cada grupo, se aplican las funciones de agregación, devolviendo un único resultado por grupo.

```
-- GROUP BY
SELECT columna1, SUM(columna2)
FROM tabla
GROUP BY columna1;
```

ntic master

UNIVERSIDAD COMPLUTENSE MADRID

4. Unit 4. Lenguaje SQL

DML (Data Manipulation Language)

- **Combinación de Tablas**
  - INNER JOIN
  - LEFT JOIN
  - RIGHT JOIN
  - FULL OUTER JOIN

5 Lenguaje DML: Combinación de Tablas

JOIN

- **Función**: Combina filas de dos o más tablas, colocando las columnas de ambas en los resultados como si vinieran de una sola tabla.

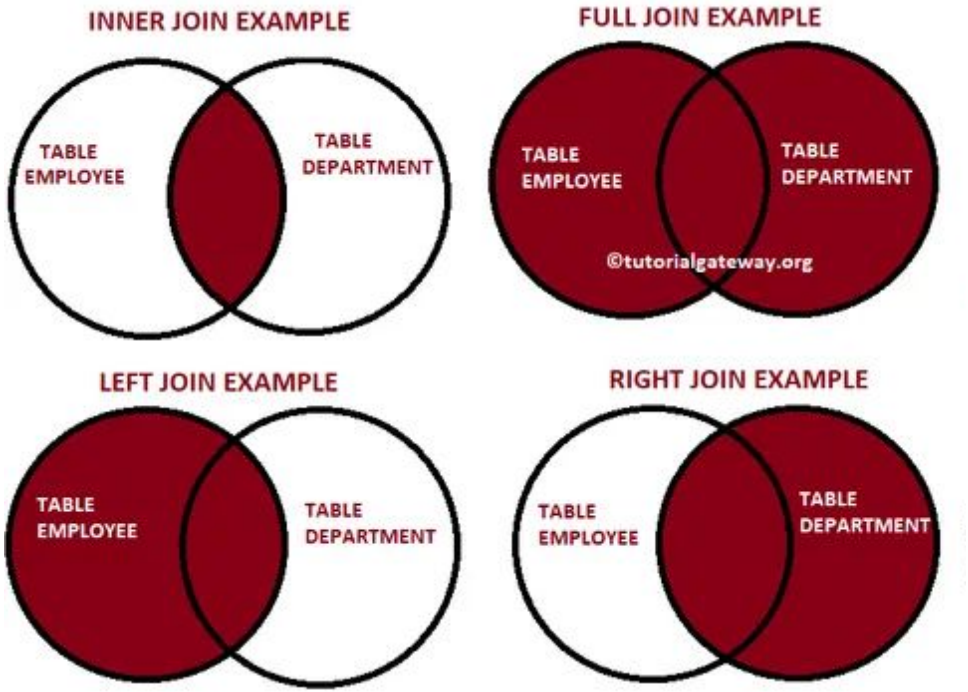
Sintaxis de JOIN

- **Sintaxis explícita** (ANSI SQL:1992):
  Recomendado y moderno. Define explícitamente el tipo de **JOIN**.
- **Sintaxis implícita**: Anticuada y no recomendada

Tipos de JOIN

- **INNER JOIN**: Devuelve filas con coincidencias en ambas tablas.
- **LEFT [OUTER] JOIN**: Devuelve todas las filas de la tabla izquierda y las coincidencias de la derecha.
- **RIGHT [OUTER] JOIN**: Devuelve todas las filas de la tabla derecha y las coincidencias de la izquierda.
- **FULL [OUTER] JOIN**: Devuelve todas las filas con coincidencias en cualquiera de las tablas.
- **CROSS JOIN**: Devuelve el producto cartesiano de las dos tablas.

5 Lenguaje DML: Combinación de Tablas



4 Lenguaje DML: Combinación de Tablas

JOIN NATURAL

En un **JOIN NATURAL**, se comparan los valores de una o más columnas en la primera tabla con los valores correspondientes en la segunda tabla. La operación se realiza automáticamente sobre las columnas que comparten el mismo nombre en ambas tablas.

En el siguiente ejemplo, se realiza un **JOIN NATURAL** entre dos tablas:

```
--JOIN NATURAL
SELECT *
FROM tabla1, tabla2
WHERE tabla1.columna_comun = tabla2.columna_comun;
```

Sin embargo este JOIN no es recomendable ya que **JOIN explícito** es preferible porque:

- **Mayor claridad**: Especifica claramente las columnas que se están uniendo, facilitando la comprensión de la consulta.
- **Más control**: Evita errores al unir columnas incorrectas, garantizando uniones precisas.
- **Mejor rendimiento**: Permite al motor optimizar mejor las consultas al saber exactamente qué columnas unir.
- **Menos errores**: Evita problemas con nombres de columnas coincidentes que podrían generar resultados inesperados.

4 Lenguaje DML: Combinación de Tablas

INNER JOIN

El **INNER JOIN** se utiliza para combinar filas de dos tablas, devolviendo únicamente las filas donde existe una **coincidencia** entre las columnas especificadas.

```
-- JOIN
SELECT t1.columna1, t2.columna2
FROM tabla1 t1
INNER JOIN tabla2 t2 ON t1.columna_comun = t2.columna_comun;
```

Función:

El **INNER JOIN** filtra las filas, devolviendo solo los datos que tienen coincidencias en ambas tablas. Se pueden usar **operadores de comparación** y otras condiciones en la cláusula **ON** para definir cómo se realiza la combinación entre las tablas.

4 Lenguaje DML: Combinación de Tablas

LEFT JOIN

Un **LEFT JOIN** devuelve **todas las filas** de la tabla de la izquierda (especificada en **FROM**) y las filas correspondientes de la tabla de la derecha (especificada en **JOIN**) que tienen coincidencias en la condición **ON**. Si no hay coincidencias, las columnas de la tabla derecha se rellenan con `NULL`.

```
-- LEFT JOIN
SELECT t1.columna1, t2.columna2
FROM tabla1 t1
LEFT JOIN tabla2 t2 ON t1.columna_comun = t2.columna_comun;
```

4 Lenguaje DML: Combinación de Tablas

RIGHT JOIN

Un **RIGHT JOIN** es lo opuesto al **LEFT JOIN**. Devuelve **todas las filas** de la tabla de la derecha (especificada en **JOIN**) y las filas correspondientes de la tabla de la izquierda (especificada en **FROM**) que coinciden con la condición **ON**. Si no hay coincidencias, las columnas de la tabla izquierda se rellenan con `NULL`.

```
-- RIGHT JOIN
SELECT t1.columna1, t2.columna2
FROM tabla1 t1
RIGHT JOIN tabla2 t2 ON t1.columna_comun = t2.columna_comun;
```

4 Lenguaje DML: Combinación de Tablas

FULL OUTER JOIN

Un **FULL OUTER JOIN** devuelve **todas las filas** de ambas tablas, independientemente de si tienen coincidencias en los campos especificados en la cláusula **ON**. Las filas que no tienen coincidencias en una de las tablas se completan con `NULL`.

```
-- FULL OUTER JOIN
SELECT t1.columna1, t2.columna2
FROM tabla1 t1
FULL OUTER JOIN tabla2 t2 ON t1.columna_comun = t2.columna_comun;
```

ntic master

UNIVERSIDAD COMPLUTENSE MADRID

4. Unit 4. Lenguaje SQL

DML (Data Manipulation Language)

○ Consultas Avanzadas

■SUBQUERIES - Subconsultas

■CASE WHEN - Condicionales

■VIEWS - Vistas

■LIKE - Patrones

■TOP - Límites

■HAVING - Filtrado de Grupos

■WINDOW FUNCTIONS - Funciones de Ventana

■CTEs - Common Table Expressions

■SET OPERATORS - Operadores de Conjunto

4 Lenguaje DML: Queries Avanzadas

SUB-QUERIES independientes

Una **subconsulta independiente** se evalúa una sola vez y su resultado es utilizado por la consulta externa. Este tipo de subconsulta es útil cuando queremos comparar un valor en la consulta externa con el resultado de una consulta interna.

Ejemplo con el Operador =

- La **consulta interna** (subconsulta) devuelve el valor máximo de `columnaB` de `tabla2` que cumple con la condición.
- La **consulta externa** compara los valores de `columnaB` en `tabla1` con el valor devuelto por la subconsulta.

```
-- Subquery Independiente
SELECT columna1
FROM tabla1
WHERE columna2 = (SELECT MAX(columna2) FROM tabla2);
```

Proceso:

1. La **consulta interna** se evalúa primero, devolviendo el valor máximo de `columna2`.
2. Luego, la **consulta externa** compara los valores de `columna1` en `tabla1` con el valor devuelto, ejecutando el filtro.

Tras evaluarse la subconsulta, la consulta externa se simplifica, permitiendo filtrar las filas de **tabla1** basándose en el valor retornado por la subconsulta.

4 Lenguaje DML: Queries Avanzadas

VIEWs

Una **vista** es una tabla virtual basada en el resultado de una consulta **SQL**. Aunque parece una tabla normal, **no almacena** los datos, solo la definición de la consulta. Las vistas se utilizan para consultar datos específicos de una o más tablas y simplificar consultas complejas.

Ventajas de las Vistas:

- **Reutilización**: Permiten almacenar **consultas complejas**, que se pueden reutilizar invocando la vista por su nombre.
- **Seguridad**: Las vistas pueden **ocultar columnas** sensibles, como salarios o números de cuentas, mostrando solo la información necesaria.
- **Simplicidad**: Facilitan la presentación de datos específicos sin necesidad de redefinir consultas complejas cada vez.

```
-- Crear Vista
CREATE VIEW nombre_vista AS
SELECT columna1, columna2
FROM tabla
WHERE condicion;
```

Las **vistas** son útiles para simplificar consultas complejas, controlar el acceso a información sensible y mejorar la claridad de los datos mostrados a los usuarios.

4 Lenguaje DML: Queries Avanzadas

```sql
-- CASE WHEN
SELECT columna1,
       CASE
           WHEN columna2 = 'valor' THEN 'Resultado1'
           ELSE 'Resultado por defecto'
       END AS nueva_columna
FROM tabla;
```

Expresiones CASE en SQL

Las expresiones **CASE** se utilizan para modificar o transformar los datos, permitiendo realizar operaciones condicionales dentro de una consulta. Un ejemplo común es clasificar un índice numérico en categorías descriptivas, como "Alto", "Medio" y "Bajo".

Tipos de Expresiones CASE:

1. **Expresión CASE simple**:
   - Evalúa un valor específico y devuelve un resultado basado en una serie de comparaciones.

2. **Expresión CASE de búsqueda**:
   - Evalúa una serie de condiciones, devolviendo un valor cuando la primera condición se cumple.

Las expresiones **CASE** son muy útiles para transformar datos basados en condiciones, y permiten manejar múltiples resultados en una sola consulta.

4 Lenguaje DML: Queries Avanzadas

Operador LIKE en SQL

El operador **LIKE** se utiliza para realizar búsquedas por **patrones**, comparando los valores de una columna con un patrón específico. Es útil para filtrar resultados en columnas de texto o fechas.

```sql
-- LIKE
SELECT *
FROM tabla
WHERE columna1 LIKE 'A%';
```

- El **patrón** debe ser compatible con el tipo de dato de la columna (alfanumérico o fecha).

Caracteres Especiales en el Patrón:

- **%**: Representa cualquier secuencia de cero o más caracteres.
  - Ejemplo: `'A%'` devolverá todos los valores que comiencen con la letra "A".
- **_**: Representa un solo carácter.
  - Ejemplo: `'A_'` devolverá todos los valores que comiencen con "A" seguido de un solo carácter.

4 Lenguaje DML: Queries Avanzadas

TOP

La cláusula **TOP** se utiliza para devolver solo las primeras **n** filas del resultado de una consulta. Es importante usarla junto con **ORDER BY** para garantizar que los resultados estén bien definidos.

- **TOP n**: Devuelve las primeras **n** filas.
- **ORDER BY**: Ordena los resultados para asegurar un conjunto determinado de filas.

Importancia de ORDER BY:

- Sin **ORDER BY**, la consulta será **no determinística**, es decir, puede devolver resultados diferentes en cada ejecución.

```
-- TOP
SELECT TOP n columna1, columna2
FROM tabla
ORDER BY columna1 DESC;
```

4 Lenguaje DML: Queries Avanzadas

HAVING

La cláusula **HAVING** es similar a **WHERE** porque ambas definen condiciones de búsqueda, pero **HAVING** se aplica a **grupos** de datos en lugar de a filas individuales. Se utiliza para filtrar resultados después de haber aplicado una agrupación con **GROUP BY**.

- **Uso común**: Generalmente se combina con **GROUP BY** para filtrar grupos basados en funciones de agregación como **SUM**, **COUNT**, **AVG**, etc.
- **Uso sin GROUP BY**: Si no se utiliza **GROUP BY**, **HAVING** trata todas las filas como un único grupo.

```
-- GROUP BY
SELECT columna1, SUM(columna2)
FROM tabla
GROUP BY columna1;
```

4 Lenguaje DML: Queries Avanzadas

UNION

El operador **UNION** combina los resultados de dos tablas en una sola, generando una tabla que contiene todas las filas que aparecen en una o ambas tablas. Por defecto, elimina los duplicados.

- Si se usa la opción **UNION ALL**, se mostrarán **todas las filas**, incluyendo duplicados.
- Las tablas deben ser **compatibles**, es decir, tener el mismo número de columnas y tipos de datos compatibles (ejemplo: **INT** y **SMALLINT**).
- **ORDER BY** solo se puede aplicar al **último SELECT** para ordenar los resultados de la unión.

```
-- UNION
SELECT columna1 FROM tabla1
UNION
SELECT columna1 FROM tabla2;
```

4 Lenguaje DML: Queries Avanzadas

INTERSECT

El operador **INTERSECT** devuelve la **intersección** de dos tablas, es decir, las filas que aparecen en **ambas** tablas.

- Solo devuelve las filas que existen en **ambas** tablas.
- Ambas tablas deben tener el mismo número de columnas y tipos de datos compatibles.

EXCEPT

El operador **EXCEPT** devuelve la **diferencia** entre dos tablas, es decir, las filas que están en la **primera** tabla pero **no** en la segunda.

- Elimina las filas que aparecen en la segunda tabla.
- También requiere que ambas tablas tengan el mismo número de columnas y tipos de datos compatibles.

```
-- INTERSECT
SELECT columna1 FROM tabla1
INTERSECT
SELECT columna1 FROM tabla2;

-- EXCEPT
SELECT columna1 FROM tabla1
EXCEPT
SELECT columna1 FROM tabla2;
```

4 Lenguaje DML: Queries Avanzadas

Common Table Expressions (CTE) con WITH

El operador **WITH** permite crear **expresiones de tabla común (CTE)** que actúan como tablas temporales dentro de una consulta.

- Facilita la **legibilidad** y **reutilización** de subconsultas complejas.
- Se define al inicio de la consulta y puede referenciarse múltiples veces.
- Ideal para **simplificar** consultas anidadas y mejorar el mantenimiento del código.

```sql
-- CTE Básico
WITH mi_cte AS (
    SELECT columna1, columna2 FROM tabla1
    WHERE condicion = 'valor'
)
SELECT * FROM mi_cte;
```

4 Lenguaje DML: Queries Avanzadas

Funciones de Ventana

Las **funciones de ventana** operan sobre un **conjunto de filas relacionadas**, también llamado **ventana**. Las filas se agrupan según ciertos criterios, como fechas, ubicaciones o categorías, y las funciones se aplican a cada grupo de manera individual sin colapsar las filas como lo harían las funciones de agregación estándar.

¿Qué es una Ventana?

- Una ventana es un **grupo de filas** definidas según ciertos criterios, como:
  - **Tiempo** (ejemplo: todas las filas del mismo mes).
  - **Ubicación** (ejemplo: todas las filas de una ciudad específica).
- Las ventanas pueden incluir **cero, una o varias filas**.
- Las funciones de ventana permiten realizar cálculos avanzados, manteniendo las filas originales en los resultados.

Componentes de las Funciones de Ventana:

1. **Fila**: La función de ventana recibe una fila individual sobre la que se realizan los cálculos.
2. **Ventana de Filas**: El grupo de filas relacionadas que define la ventana. Puede ser la tabla completa o un subconjunto definido por condiciones específicas.

4 Lenguaje DML: Queries Avanzadas

Sin Partición

Sin Partición ni Ordenación:

- **OVER()** sin partición ni ordenación aplica la función de ventana a **toda la tabla**, devolviendo la misma suma total del salario para cada fila.

Con Ordenación (y Sin Partición)

- **ORDER BY** dentro de la cláusula **OVER()** primero ordena los empleados por **salario** de mayor a menor.
- Luego, para cada fila, devuelve el **salario acumulado** de todos los empleados hasta esa fila.

Proceso:

1. **Sin especificar partición ni orden**: El cálculo de la función agregada se realiza sobre **toda la tabla**.
2. **Con ordenación**: Se aplica la agregación después de ordenar los datos, lo que permite ver el efecto de la acumulación o el rango dentro de un conjunto ordenado.

4 Lenguaje DML: Queries Avanzadas

Particiones en Funciones de Ventana

Para calcular las **funciones de ventana**, el conjunto de resultados de una consulta se divide en una o más **particiones**. Cada partición está formada por las filas que comparten el mismo valor para los términos definidos en la cláusula **PARTITION BY**. Si no se especifica **PARTITION BY**, todo el conjunto de resultados se considera como una única partición.

Ejemplo con Partición

- **PARTITION BY** agrupa filas dentro de la ventana, y las funciones de ventana se aplican de forma independiente a cada grupo.
- Si no se utiliza **PARTITION BY**, el cálculo de la función de ventana se realiza sobre **toda la tabla**.

Proceso

- **Reinicio de la función**: El cálculo de la función se **reinicia** para cada partición. En este ejemplo, la suma de salarios se calcula y se acumula **separadamente para cada departamento**.
- Si no se hubiera utilizado **PARTITION BY**, el cálculo de la suma se habría hecho para **toda la tabla** sin agrupar por departamentos.

4 Lenguaje DML: Queries Avanzadas

1. **Función Sin Partición ni Ordenación**:
   - **Aplicación**: El cálculo de la suma se realiza sobre **toda la tabla**.
   - **Resultado**: Devuelve el **mismo valor** de la suma total del salario para cada fila.
2. **Función con Ordenación**:
   - **Aplicación**: Ordena las filas por **salario descendente** antes de aplicar la función.
   - **Resultado**: Devuelve un **salario acumulado** que se va sumando fila por fila a medida que recorre el conjunto ordenado.
3. **Función con PARTITION BY y Ordenación**:
   - **Aplicación**: La función se aplica de manera **independiente** para cada grupo (departamento en este caso) definido por **PARTITION BY**.
   - **Resultado**: Calcula el **total acumulado del salario** dentro de cada departamento, reiniciando el cálculo para cada partición.

Resumen:

- **Sin Partición**: La función se aplica a toda la tabla sin agrupar.
- **Con Ordenación**: La función se aplica sobre el conjunto de filas ordenadas.
- **Con Partición**: La función se aplica **por grupo** (partición) y se reinicia para cada partición.

4 Lenguaje DML: Queries Avanzadas

Funciones de Ventana en SQL

```sql
-- Función de ventana sin partición ni ordenación: Suma total del salario para toda la tabla
SELECT nombre,
       SUM(salario) OVER() AS total_salarial
FROM empleados;

-- Función de ventana con ordenación: Salario acumulado para cada empleado
SELECT nombre,
       salario,
       SUM(salario) OVER(ORDER BY salario DESC) AS acumulado_salarial
FROM empleados;

-- Función de ventana con PARTITION BY: Salario total por departamento
SELECT id_departamento,
       nombre,
       SUM(salario) OVER(PARTITION BY id_departamento ORDER BY salario DESC) AS total_salarial_departamento
FROM empleados;
```

ntic master



UNIVERSIDAD
COMPLUTENSE
MADRID